

TIME WILL TELL: TIMING SIDE CHANNELS VIA OUTPUT TOKEN COUNT

by

Tianchen Zhang

A thesis submitted in conformity with the requirements
for the degree of Master of Applied Science

Department of The Edward S. Rogers Sr. Department of Electrical and Computer
Engineering
University of Toronto

© Copyright 2025 by Tianchen Zhang

Tianchen Zhang
Master of Applied Science

Department of The Edward S. Rogers Sr. Department of Electrical and Computer Engineering
University of Toronto
2025

Abstract

This thesis presents a novel timing side channel for large language model (LLM) inference, enabling an adversary to extract sensitive information based on the number of output tokens in the response. Our attack relies on the observation that the autoregressive generation in LLMs enables a network-based adversary to reliably estimate the output token count from the LLM generation time. For task-specific LLMs, the output token count may correlate with sensitive attributes in users' inputs. We construct attacks in two common LLM tasks, enabling an adversary to recover the output class in classification tasks and the target language in machine translation.

With open-source models, our experiments demonstrate that a network-based adversary can achieve with an average attack success rate (ASR) of 78% in classification and 80% in translation. Notably, the attack also proves effective on close-sourced OpenAI's GPT-4o. Finally, we propose tokenizer-, system-, and prompt-based mitigations against the output-token-count side channel.

Contents

1	Introduction	1
2	Background	4
2.1	Side-channel Attacks	4
2.1.1	Side-Channel Attacks Through Microarchitectural Vulnerabilities	4
2.1.2	Side-Channel Attacks on Machine Learning	5
2.2	LLM Autoregressive Decoding	5
2.3	Few-shot Prompting	6
2.4	Tokenization	6
3	Related Work	8
3.1	Timing Side Channels in Machine Learning Algorithms	8
3.2	Side-channel Attacks on LLMs	9
4	Timing Side Channel Mechanism	10
4.1	Threat Model	10
4.2	Recover Output Token Count from LLM Execution Time	11
4.3	Token Count Recovery on Local LLM Servers	11
4.4	Token Count Recovery from GPT-4o on Remote Servers	12
4.5	Exploiting Token Count Side Channels	14
5	Attack on Few-Shot Classification Tasks	15
5.1	Application Scenario	15
5.2	Attack Design	16
5.3	Attack Implementation	16
5.4	Experiment Setup	16
5.5	Profiling Results	18
5.5.1	Class Profiling for Explanation Length Bias	18
5.5.2	Attack Success Rate with Output Token Count	18
5.6	Timing Attack on Local LLM Servers	20
5.7	Timing Attack on GPT-4o on Remote OpenAI Servers	20
5.8	Additional Results	22
5.8.1	Dataset Setup and Results for Motivation Examples in Classification Attack	22
5.8.2	Varying Number of Few Shot Examples	22

5.8.3	Alternative Model Architectures and Sizes	22
5.8.4	Approximate Prompt	23
5.8.5	Ablation studies	24
6	Attack on Translation Workloads	25
6.1	Application Scenario	25
6.2	Attack Design	25
6.3	Attack Implementation	26
6.4	Experiment Setup	27
6.5	Profiling Results	28
6.5.1	Class Profiling	28
6.5.2	Attack Success Rate with Output Token Count	28
6.6	Timing Attack on Local LLM Servers	30
6.7	Timing Attack on GPT-4o on Remote OpenAI Servers	31
6.8	Additional Results	31
6.8.1	Varying Attacker Samples	31
6.8.2	Alternative Source Languages	32
6.8.3	Alternative Test Dataset	33
6.8.4	Ablation Study	33
7	Potential Defences	36
7.1	Prompt-level Defence	36
7.2	Tokenizer-level Defence	36
7.3	System-level Defence	37
8	Discussion	38
9	Conclusion	39
A	Appendix	40
A.1	Classification Tasks from Natural Instruction Dataset	40

List of Tables

4.1	Normalized Root Mean Square Error (NRMSE) for the Output Token Count Predicted using Observed LLM Generation Times	12
5.1	ASR (%) of Augmenting class prompts for different model architectures and sizes . .	19
5.2	ASR (%) of Gemma-9B-IT with Augmenting class prompts with timing information	20
5.3	ASR (%) for two motivation examples, with Augmenting 3-shot examples and direct access to output token count.	22
5.4	ASR (%) for approximate prompts, with Augmenting 3-shot examples (difference of ASR from the original prompt in the training set across all model predictions in brackets).	23
5.5	ASR (%) as the temperature varies, for the augmenting 3-shot examples. Gemma stands for Gemma2-9B-IT model.	24
5.6	ASR (%) only for results with only correct model predictions, with Augmenting 3-shot examples (difference with ASR across all model predictions in brackets).	24
6.1	Translation performance (BLEU scores) for different models, M2M100, MBart50, Tower.	28
6.2	Target Languages Used to Test our Attack, Grouped by Language Family	28
6.3	Language prediction ASR in the network scenario with timing information for 50 samples (Difference between timing and output token ASR shown in brackets) . . .	30
6.4	ASR (%) for recovering target language with source languages other than English for Tower and M2M100 model.	32
A.1	Classification Tasks from Natural Instructions Dataset Used in Sections 6	40

List of Figures

1.1	Example of our side-channel attacks on text sentiment classification with an LLM. (a) For a given text input, the LLM outputs the classified sentiment and also provides an explanation that can vary in length. (b) Our token-count side-channel attack leaks the output class (sentiment) based on the bias in the output token counts, or by measuring the execution time, as a proxy for the output token count.	2
4.1	Threat model. We assume the attacker is network-based, and cannot inspect the contents of the encrypted input or output, but can monitor the overall response time and length of the entire encrypted input or output.	10
4.2	Time per Output Token (TPOT) Profiling for GPT-4o.	12
4.3	System Prefilling Delay Profiling for GPT-4o responses.	13
4.4	Correlation between output token count and response time for GPT-4o model. Attacker’s predictions for token counts (green line) have an NRMSE of 0.465 compared to victim’s actual token counts (blue dots).	14
5.1	Prompt template for our tasks, using the Predict-then-Explain (P-E) format. The system prompt consists of the task description and the input examples for each class (e.g., one-shot example per class). If the few-shot examples have biased explanation lengths (shorter for class A), it is possible the response with output class A also has a shorter explanation.	17
5.2	Output Token Distribution for Task-145, with Unbiased, Augmenting and Diminishing examples used as few-shot examples (three-shot per class). The Unbiased is chosen as “Long-Long”.	18
5.3	ASR (%) across all tasks for Gemma2-9B, with unbiased and biased examples with 3-shot examples per class.	19
5.4	Average ASR (%) across tasks for GPT-4o on different days	21
5.5	ASR (%) as the number of few-shot examples is varied from 1-shot to 2-shot to 3-shot, for Gemma2-9B, with augmenting examples.	23
6.1	Overview of Attack on Translation. A user translates text in a source language (e.g., English) to any target language; the goal of the attack is to leak this target language. By profiling output token density and output/input bytes ratio and observing these for a given translation, the attacker can leak the user’s target language.	26
6.2	Profile using a Gaussian Mixture Model on a 2D decision space for the Tower model with the source language as English.	29

6.3	Attack success rates (ASR) for the attack leaking the output languages for different translation models, using English as the source language.	29
6.4	ASR (%) for recovering target language in GPT-4o using response time averaged across ten repeated requests with concurrent profiling.	31
6.5	Attack success rate (ASR) for the translation attack on the Tower model as the number of sampled requests from the user increases from 1 to 10 to 30 to 50 samples (default = 50).	32
6.6	Attack success rate (ASR) for the translation attack on Tower model using EuroParl dataset as test set (retaining Flores as training set).	33
6.7	Ablation study showing ASR (%) when only output token density or output-input bytes ratio is used as the side channel, compared with attack using combined channel, for Tower model with source language English.	34
6.8	Ablation study for ASR (%) using just token density or just output-input length ratio for M2M100 model, versus combined channel (default).	34
6.9	Ablation study for ASR (%) with just token density or just output-input length ratio for MBart50, versus combined channel (default).	35

Chapter 1

Introduction

Large language models (LLMs) [1,2] have rapidly gained adoption in web applications like OpenAI’s ChatGPT [3], Microsoft’s Copilot [4], and Anthropic’s Claude [5], due to their ability to generalize across diverse tasks like machine translation and text classification. However, their widespread deployment has raised critical privacy concerns, with prior works showing attacks like membership inference [6,7], training data extraction [8], and prompt injection [9], posing substantial privacy risks for the data used to train these models.

Additionally, users interacting with LLM-based applications at inference time also face privacy risks, as the outputs of these interactions can contain sensitive and private information about them. For instance, in applications such as disease diagnosis or loan adjudication relying on LLMs to analyze and classify text, an eavesdropper who can intercept their outputs could misuse the information, targeting users with predatory loans [10] or fraudulent medical treatments [11,12]. Similarly, LLMs increasingly power machine translation services like Google Cloud’s adaptive translation [13] and DeepL [14]. A malicious actor, analyzing a user’s translation outputs, could expose a user’s preferred or native language, potentially revealing ethnicity or nationality, both classified as personally identifiable information (PII) under GDPR [15]. Unscrupulous ISPs could use such an attack to infer and sell data about their customers to third-party advertisers [16]. This information could even be weaponized by oppressive regimes to track and monitor ethnic or religious minorities [17], as well as foreign journalists [18].

While communications with LLMs might be encrypted, preventing a network attacker from directly reading LLM outputs, in this paper, we show that this is insufficient to protect the leakage of sensitive information regarding LLM outputs. By exploiting a timing-side channel on the latency of the LLM response, we show that a passive network attacker can still infer sensitive information, such as the output class in classification tasks, or the output language in translation tasks.

Recent work has used side channels in LLM systems to breach user privacy at inference time in LLMs. For instance, LLM keylogging attack [19] was able to infer 27% of the LLMs outputs using the character length of each token. Other work exploited LLM performance optimizations, such as speculative decoding [20] and shared semantic caches and KV caches [21], to leak user prompts via variations in packet sizes and timing.

While these previous works depend on access to individual encrypted tokens [19], or rely on specific system optimizations of LLMs [20,21], we are the first to demonstrate a general timing

side channel that leaks information about the number of tokens in the LLM output, and show how this side channel can be exploited to glean sensitive information. Further, as our timing side channel is a product of the autoregressive generation mechanism used in *all* modern transformer-based LLMs [1, 2], this makes many modern LLMs vulnerable to this side channel. Autoregressive generation means that such LLMs generate output tokens one at a time, with an approximately constant delay per token, making the side channel easy to exploit.

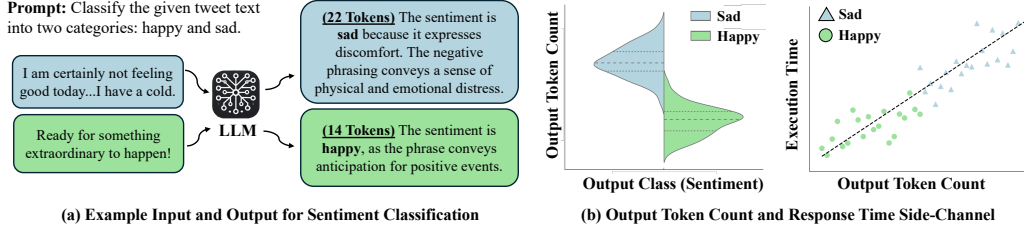


Figure 1.1: Example of our side-channel attacks on text sentiment classification with an LLM. (a) For a given text input, the LLM outputs the classified sentiment and also provides an explanation that can vary in length. (b) Our token-count side-channel attack leaks the output class (sentiment) based on the bias in the output token counts, or by measuring the execution time, as a proxy for the output token count.

We also demonstrate that output token count, as recovered by our timing side channel, can be combined with other task-specific biases of LLMs to infer sensitive attributes of the LLM outputs. For instance, as shown in Figure 1.1, an LLM performing classification and generating explanations may produce longer output responses for one class over the other. Our experiments show that such biases exist in a task-dependent manner across tens of tasks in all LLMs we profile (Llama3, Gemma2, GPT-4o), enabling a network adversary, observing network delays, to infer the output class without seeing the plaintext output. This attack can reveal sensitive information in the aforementioned applications, including loan approval and diabetes diagnosis, as demonstrated in our simulated scenarios in Section 5.8.1. Similarly, we show how biases in tokenizers due to resource-poor languages [22, 23], that cause variations in the token density (output plaintext bytes per token) across different languages, can be combined with our output token timing-side channel by a network adversary to infer the language of the output of an LLM translation task.

Effectiveness of our Attacks. We find that by measuring input-to-output latency, a network attacker can infer the token count of the LLM output with less than 0.068 Normalized Root Mean Square Error (NRMSE) on a local network on Tower, M2M100, MBART50 and Gemma2-9B. We further show that our side channel works on commercial LLMs, such as GPT4o, running on remote OpenAI servers, where we find an adversary can still recover the output token count with an NRMSE of 0.316, despite having no knowledge of the internal implementation of the model, or the infrastructure serving the model.

When we study the use of our side channel to recover the output of classification tasks, we find that a network adversary is able to achieve an attack success rate (ASR) of 77.6% for Gemma2-9B. Similarly, when used to infer the output language of an LLM translation task, an attacker using our side channel is able to identify over 12 languages across three different multilingual models, Tower, M2M100, and MBart50, with an average ASR of 82.5%, 77.2%, and 80.5%, respectively. Moreover, languages with distinctive morphology, such as Chinese, Russian, Hindi, Korean, and Arabic, reach almost 100% ASR. When targeting black-box API access of GPT-4o, the attacker can still achieve

an ASR of 73.8% and 77.2% on classification and translation tasks, respectively. When the adversary has access to multiple repeated requests (10 in our experiment), these ASRs can be further increased to 85.0% and 81.6%.

Summary of Contributions:

1. **New Timing Side Channel via Output Token Counts:** We identify a new timing side channel in LLMs due to variations in output token counts and the autoregressive nature of LLMs. These variations, correlated with task-specific sensitive attributes, enable attackers to infer private information using execution time measurements.
2. **Highlight Token-Count Bias in Text Classification:** We demonstrate that inherent biases in the token counts of generated explanations across classification tasks can leak sensitive output classes. We show that few-shot prompting can amplify these biases, increasing the attack success rate of token-count side channels.
3. **Demonstrate Tokenizer Bias is a Privacy Risk:** We show that language-based differences in token density and output token counts can leak language preferences through our side channel, reframing these biases as a privacy risk rather than solely a fairness issue.
4. **End-to-End Remote Attacks over the Network:** We show that timing attacks are successful over the network with a local LLM server, in leaking output language in translation tasks (75% to 83% ASR) and output class in text classification (with 77% ASR). We further show that this attack remains equally effective for black-box API access of proprietary LLMs running on remote servers many hops away, where the timings can be disturbed by server load, network noise, and LLM inference optimizations (achieving 84.9% ASR for classification tasks and 81.6% for translation).

The thesis is organized as follows. Chapter 2 introduces the background on side-channel attacks and key concepts relevant to our attack methodology. Chapter 3 reviews related work on timing side channels in machine learning, as well as side-channel attacks targeting LLMs. In Chapter 4, we define the threat model for a network-based adversary and present our approach for inferring output token counts from remote execution timings. Chapter 5 presents an end-to-end attack on classification tasks, demonstrating effectiveness in both local server settings and proprietary black-box APIs. Chapter 6 extends the attack to translation tasks, showing its generality. To mitigate this side channel, we propose prompt-, tokenizer-, and system-level defences in Chapter 7. Finally, we discuss broader implications and limitations in Chapter 8 and conclude in Chapter 9.

Chapter 2

Background

2.1 Side-channel Attacks

Side-channel attacks originated in cryptography, exploiting unintended information leakage from the system implementation rather than vulnerabilities in the algorithm itself. These attacks leverage subtle signals such as timing variations [24, 25], power consumption [26], and electromagnetic emissions [27]. With continued advancements in the field, side-channel techniques have extended beyond cryptographic contexts to compromise the privacy of systems involving privileged memory access, website fingerprinting, and machine learning models and data.

2.1.1 Side-Channel Attacks Through Microarchitectural Vulnerabilities

Microarchitectural side channels constitute a large spectrum of side-channel attacks. Early works exploit the cache hierarchy by exploiting timing differences between cache hits and misses. Liu et al. [28] present the *PRIME+PROBE* attack, targeting the shared Last-Level Cache (LLC) across cores. In the *PRIME* phase, the attacker fills specific cache sets with its data and enters an idle state, allowing the victim to execute. In the *PROBE* phase, the attacker reloads each cache set and measures access times. Longer access times indicate eviction, suggesting the victim accessed that cache set. The *FLUSH+RELOAD* attack [29] is a variant of *PRIME+PROBE*, offering finer granularity by targeting individual memory lines rather than cache sets. While it requires shared memory pages between the attacker and victim, this constraint enables lower noise and more precise observations.

Microarchitectural optimization techniques, such as speculative and out-of-order execution, have enabled a new class of side-channel attacks beyond traditional cryptographic contexts. Meltdown attack [30] exploits out-of-order execution to access arbitrary kernel memory from user space. Attackers can trick CPUs into accessing privileged memory and performing further computation on the value to access a cache line. Even if the direct computation is later reversed, it creates a data-dependent trace in the cache that can be recovered with Flush+Reload, allowing the attacker to infer the secret value stored in privileged memory. Foreshadow [31] further presents that such Meltdown-like attacks can be applied to break the isolation of Intel SGX’s secure enclaves from unprivileged user space. Spectre attack [32] exploits speculative execution and branch predictors to execute privileged instructions speculatively and leave a data-dependent trace similar to Meltdown. Compared

to Meltdown attacks, Spectre affects a broader range of CPU architectures and continues to evolve with new variants, making it more challenging to mitigate. The widespread impact of both Spectre and Meltdown has significantly heightened interest in microarchitectural security vulnerabilities and beyond.

2.1.2 Side-Channel Attacks on Machine Learning

With the rapid advancement of machine learning, side-channel attacks have increasingly shifted toward targeting machine learning models and data, emerging as a prominent alternative to traditional attacks on cryptographic algorithms.

With growing interest in microarchitectural side channels, many attacks have expanded to target GPUs to extract information about model architectures. Naghibijouybari et al. [33] use the co-location of attacker and victim in the resource space of Nvidia GPUs to interleave execution. The side channels are then exploited through memory allocation, program counters and time measurement to reconstruct the neural network structure. Wei et al. [34] instead target GPU context-switching penalties to recover DNN attributes, including layer architectures and hyperparameters. By disabling the Multi-Process Service and forcing context switches between victim and attacker, they achieve significantly higher sampling rates without requiring co-location within a single Streaming Multiprocessor.

Besides exploiting microarchitectural vulnerabilities, several attacks directly target the properties of machine learning algorithms. Cache Telepathy [35] exploits the fact that DNN heavily relies on tiled GEMM, and DNN architecture determines the number and the dimension of GEMM calls. By using PRIME+PROBE and FLUSH+RELOAD, an attacker can effectively observe the cache usage of the tiled GEMM and deduce the number of layers and configurations. DeepSniffer [36] extracts architectural hints through memory access volumes, kernel execution time and input/output data ratios to reconstruct runtime layer sequences, topologies, and dimensions. DeBenedetti et al. [37] extend side-channel analysis to large language model (LLM) systems, revealing vulnerabilities in various pipeline stages across training data filtering, input preprocessing, output filtering, and query filtering.

2.2 LLM Autoregressive Decoding

Autoregressive generation in LLMs generates tokens sequentially, with each token depending on all prior tokens. For a sequence $X = (x_1, x_2, \dots, x_t)$, the next token x_{t+1} is predicted using the probability distribution $P(x_{t+1} \mid x_1, x_2, \dots, x_t)$. This sequential dependency limits generation to one token per iteration.

LLM token generation has two phases: *prefill* and *decode*. The *prefill* stage processes the input prompt and generates the Key-Value (KV)-cache. This stage is highly parallelizable, effectively utilizing the GPU’s computational resources. The *decode* stage occurs sequentially as each decoded token depends on all the ones that precede it. This process is more memory-bound, requiring a significant portion of time moving the data rather than doing computation. Thus, the decode stage takes the most significant proportion of the overall generation time compared to the prefill stage. Additionally, the Time Per Output Token (TPOT) remains relatively constant, increasing only

slightly with output length. As a result, the generation time (T) for a sequence (X_n) of n tokens grows linearly with (n), *i.e.*, $T(X_n) \propto n$.

LLM-based applications operate in either *streaming* or *non-streaming* modes. In *streaming* mode, used by interactive applications like chatbots [4, 38], tokens are delivered to the user immediately as they are generated. This enables adversaries to observe fine-grained TPOT and token counts. In *non-streaming* mode, often used for text processing, only the total generation time is observable. In our scenario, we demonstrate that our attacks are effective even in non-streaming mode, using only the overall execution time to recover the output token count.

2.3 Few-shot Prompting

LLMs are increasingly applied to text classification tasks, such as sentiment analysis [39], intent detection [40], and topic classification [41], where task-specific data is often required. Traditional methods like fine-tuning demand large labelled datasets and significant compute resources [42], making them impractical for many applications.

Few-shot prompting [43] offers an efficient alternative by providing a small number of task-specific examples in the input prompt. This enables the model to perform the task without expensive fine-tuning, leveraging its ability to learn in context from the examples provided. This approach is useful in scenarios where data is scarce or rapidly evolving and can provide performance comparable to fine-tuning.

Research on few-shot prompting has explored biases in the text classification output and how provided demonstrations have helped improve the performance. For example, prior works [44] identify that answers from the LLM can be biased towards a certain direction. Furthermore, prior works show that including explanations and optimal demonstration examples in prompts can improve accuracy [45–47].

In contrast, we examine whether text classification tasks have natural biases in output lengths, and whether few-shot prompting can unintentionally enhance signals in output lengths, affecting the side channels leaking output classes.

2.4 Tokenization

Tokenization is the process of splitting text into smaller units, or tokens, for processing by language models. Subword tokenizers, such as Byte Pair Encoding (BPE) [48], WordPiece [49], and SentencePiece [50], have become standard in LLMs since the introduction of the transformer architecture [1]. These methods strike a balance between a fixed vocabulary size and the ability to handle rare or unseen words by leveraging meaningful subwords that reflect language morphology.

Subword tokenizers exhibit language-dependent bias in their encoding efficiency. With BPE tokenizers used in English-centric LLMs (Llama, GPT, Mistral etc.), tasks in languages with less training data (*i.e.* resource-poor) require up to 13 times more tokens than English due to over-fragmentation [22, 23]. The imbalance in the training datasets for these languages results in sub-optimal vocabulary allocation, reducing compression rates [51] and increasing latency. Although tokenizers in multilingual models (*e.g.*, M2M100 [52], BLOOM [53]) exhibit less bias, several languages still require token counts $2.5\times$ higher than English.

While there are proposed methods to address these issues, including improving sampling efficiency [54], better vocabulary allocation [55], and minimizing over-fragmentation in multilingual settings [56], language bias remains pervasive.

Chapter 3

Related Work

3.1 Timing Side Channels in Machine Learning Algorithms

Similar to our work, numerous studies have exploited timing variations arising from inherent properties of machine learning algorithms as effective side channels for information leakage without relying on microarchitectural vulnerabilities.

As one of the earliest works in this area, Duddu et al. [57] attempt to infer model architectures using timing information. They utilize the sequential execution property of neural networks, where total execution time approximates the sum of individual layer runtimes. By querying a black-box model and measuring its execution time, the attacker infers the network depth, thereby significantly reducing the search space for constructing a substitute model through knowledge distillation. In addition to model architecture, training data is a common target in side-channel analysis, typically through membership inference attacks. Ali et al. [58] exploit timing variations in the tokenization process of Named Entity Recognition models. They observe that out-of-vocabulary words incur higher processing latency, enabling a timing-based membership inference attack. With black-box access and timing measurements, the attacker can determine whether specific inputs, such as passwords, are present in the training dataset, achieving near-perfect accuracy in their experiments.

Beyond model architectures and training data, several works aim to infer sensitive attributes from user inputs by exploiting timing variations correlated with specific input features. Shukala et al. [59] leverage non-constant-time branching behaviour in PyTorch’s max pooling layers to leak class predictions from convolutional neural networks. By relying solely on execution time measurements, the attacker can infer predicted classes and perform membership inference attacks, even when differential privacy mechanisms are in place. Without relying on implementation flaws, Akinsanya et al. [60] exploit the execution time variations in adaptive neural networks, where the computational pathway depends on the input complexity, influencing total runtime. They demonstrate that such complexity may correlate with data clusters of certain classes, allowing the attacker to infer vulnerable class labels with high accuracy using only execution time observations.

In contrast to prior works, we target LLMs by exploiting algorithmic timing variations arising from autoregressive decoding, a fundamental mechanism present in all LLMs. These variations yield pronounced and stable timing signals that are inherently more robust to system-level noise. Similar to [59] and [60], our focus is on inferring sensitive input attributes, rather than recovering model

parameters or performing membership inference on training data.

3.2 Side-channel Attacks on LLMs

More recently, a growing line of works has focused on the inference time of LLMs, targeting them through various forms of side-channel attacks.

Tokenization has emerged as a prominent side channel through which information can be leaked based on how inputs are decomposed into tokens. DeBenedetti et al. [37] pioneered side-channel attacks in LLM systems, with their input preprocessing attack closest to our work, leveraging the tokenization process and limited context window size to leak out uncommon strings in the tokenizer’s vocabulary. In contrast, our approach exploits linguistic morphological differences and biases in vocabulary resource allocations across languages to leak the language itself, rather than targeting any specific strings in the tokenizer’s vocabulary. LLM keylogging attacks [19] exploit the token side channel through their lengths. They utilize network packet analysis in streaming LLMs to extract the length of output tokens (sizes in bytes). Subsequently, they fine-tune a model to predict the next token based on extracted sequences of token lengths, to extract the content of the output response. In contrast, our attack works on even non-streaming mode LLMs, where the attacker cannot infer output token lengths. Our approach focuses on leaking sensitive input attributes based on the number of output tokens without relying on individual token lengths.

Other works exploit side channels arising from performance optimizations. Wei et al. [20] introduce a new side channel that exploits speculative decoding in LLMs for latency optimization. They observe that input-dependent token count variations introduced by correct and incorrect speculative decoding can be used to fingerprint and leak private user inputs, as well as leak private and confidential data used for generating predictions. These variations in token counts are observed based on variations in packet sizes in streaming LLMs. A similar approach is also used in Wiretapping LLMs [61] to reveal private attributes in LLM-based medical and financial applications. Song et al. [21] and Zheng et al. [62] propose similar timing attacks exploiting the shared key-value (KV) and semantic caches on LLM serving platforms, used for optimizing the input processing latency. They demonstrate Prompt Stealing Attacks by incrementally searching on static system prompts or known documents in retrieval-augmented generation (RAG) systems. Moreover, by utilizing the semantic cache, they can recover semantically similar retrieved documents from other users to infer private attributes. In contrast to these approaches, which rely on signals from performance optimizations that can be turned off, our method exploits timing signals from the autoregressive decoding process inherent in all transformer-based LLMs, making it significantly harder to mitigate. Moreover, unlike [21] and [62] which utilize timing signals from the prefill phase, our work is grounded in the timing variations from the autoregressive decoding phase, which makes up the dominant part of the LLM generation time; thus our attack exploits stronger timing variations and signals compared to prior works.

Chapter 4

Timing Side Channel Mechanism

4.1 Threat Model

Attack Scenario. As shown in Figure 4.1, we study a typical scenario where an LLM-based application runs on the server and the user interacts with it over the internet. The network packets are encrypted, but their plaintext lengths remain unchanged and are observable by the attacker, as in prior work [19]. However, unlike prior work, our attack does not require the LLM to operate in streaming mode; it remains effective even when the LLM runs in non-streaming mode, delivering responses all at once rather than token by token.

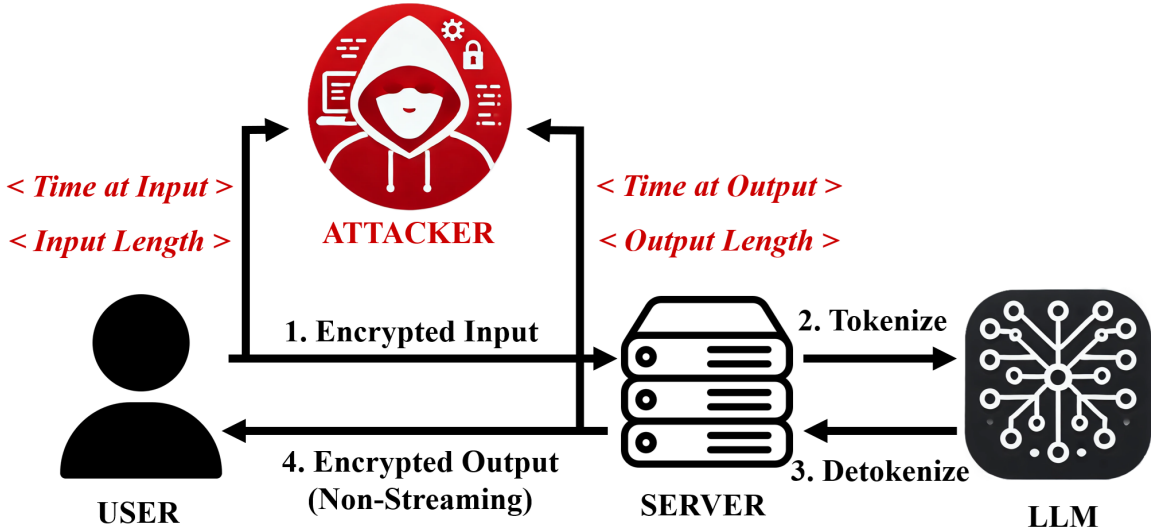


Figure 4.1: Threat model. We assume the attacker is network-based, and cannot inspect the contents of the encrypted input or output, but can monitor the overall response time and length of the entire encrypted input or output.

Attacker Capabilities. We assume a network-based attacker capable of monitoring the timing of the victim’s request to the LLM-based application and the response. We assume the attacker knows the task for which the user uses the LLM-based application. Additionally, the attacker can also interact with the LLM application as a regular user, allowing the attacker to obtain profiles of how the LLM processes requests. We assume the attacker can obtain the LLM response time, i.e.,

time from the input being sent to the server to the response being received by the user. The network attacker is limited to observing a non-streaming LLM server and approximates the output token count using the LLM response time. We show that the attack is effective regardless of whether the attacker targets an open-source model running on a local server or a proprietary model running on a remote server many hops away (e.g., OpenAI’s servers).

Attacker’s Goal. The attacker’s goal is to leak private information about the user, such as the target language in translation tasks or the output class in classification tasks. The attacker seeks to leverage the output token count timing side channel to leak private attributes correlated with the token count with high accuracy.

4.2 Recover Output Token Count from LLM Execution Time

For our timing side channel to be effective, we first need to confirm that an attacker can reliably recover the number of output tokens using the LLM generation time when running on a local server. We conducted a series of experiments on a machine equipped with an Nvidia RTX A6000 GPU with 48GB of GPU memory for all the open-source models used later in our experiments. All models utilize the HuggingFace implementations without additional latency optimizations, and the batch size is fixed at one to ensure that each request is processed independently. The time measurements are taken on classification tasks and translation tasks, which we later use in Chapter 5 and Chapter 6. We measure the token recovery error in two phases: a *profiling* phase and an *attack* phase.

Profiling Phase. The attacker interacts with the LLM, sending it queries, and recording the generated token count and execution time for each request. They then fit a linear function to estimate the token count from execution time.

Attack Phase. For the attack, the adversary monitors the LLM request and response packets between the victim user and the LLM server over the network, and gets the time between the request and response, of which the LLM generation time dominates. Using the observed LLM generation time, the adversary can predict the victim’s output token count reliably and thus, also learn any user private information that is correlated with the output token count.

We use Normalized Root Mean Square Error (NRMSE) over the mean predicted values to quantify the error between the predicted token counts and the actual ground truth and demonstrate that this approach enables successful token count recovery in settings with a local LLM server running open-source LLMs and even a commercial LLM running on a remote proprietary server.

$$\text{NRMSE} = \frac{\sqrt{\frac{1}{n} \sum_{i=1}^n (\text{Actual Output Token Count} - \text{Predicted Output Token Count})^2}}{\frac{1}{n} \sum_{i=1}^n \text{Actual Output Token Count}} \quad (4.1)$$

4.3 Token Count Recovery on Local LLM Servers

We first demonstrate the token count timing side channel on a local LLM server. Here, the LLM is hosted on a server on the same local network as the victim, and the attacker mimics a compromised router, able to snoop the packets on this network. We perform the profiling and attack phase on three different models specialized in multilingual translation: Tower, M2M100, and MBart50, as well as one general-purpose LLM, Gemma2. We perform this for at least 1000 requests from a victim

for each model, simulating typical translation requests or classification task requests (e.g., for loan approval).

As shown in Table 4.1, across all four models running on a local LLM server, the NRMSE for the attacker’s predicted output token counts is at most 0.068, with the Tower model achieving a near-perfect prediction (NRMSE of 0.003). These results demonstrate successful recovery of the output token counts from generation times with our linear predictor, and a strongly linear relationship between them.

Table 4.1: Normalized Root Mean Square Error (NRMSE) for the Output Token Count Predicted using Observed LLM Generation Times

	Tower	M2M100	MBart50	Gemma2
NRMSE	0.003	0.061	0.068	0.022

4.4 Token Count Recovery from GPT-4o on Remote Servers

Real-world commercial LLMs are likely to demonstrate significant variations in generation latencies, due to variations in network latency, server loads, and undocumented inference optimizations. To demonstrate the applicability of our timing side channel to a commercial LLM, we extend the attack to a commercial GPT-4o model running on remote OpenAI servers, replacing the local LLM server. We also assume the victim and the attacker are far apart in different networks: the victim runs on the AWS’s Ohio data center (5Gbps), while the attacker operates on our local academic network (500Mbps). To account for the difference in the locations, the response times measured by the attacker are adjusted according to the ping latency of the OpenAI server.

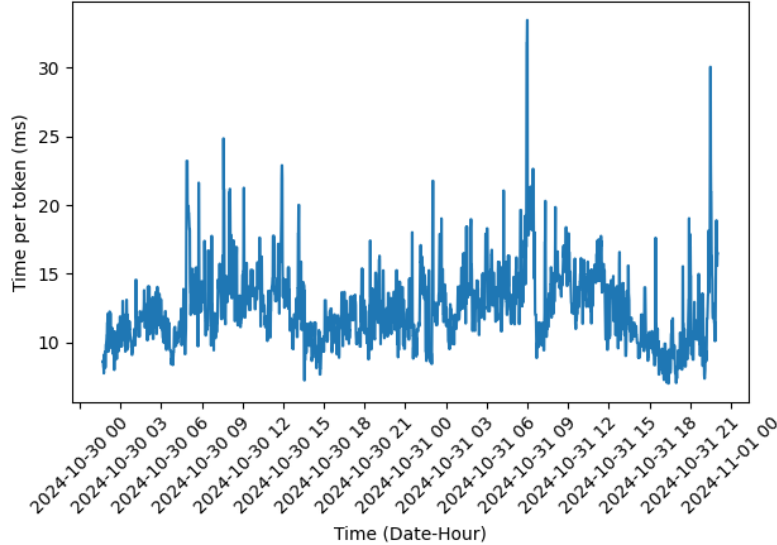


Figure 4.2: Time per Output Token (TPOT) Profiling for GPT-4o.

We observe that variations in OpenAI server loads may introduce significant noise in the observed LLM generation times, masking the relationship with token counts as shown in Figure 4.2 and Figure 4.3. To adjust for varying server load, we propose a “concurrent profiling” method and a “median aggregation” method.

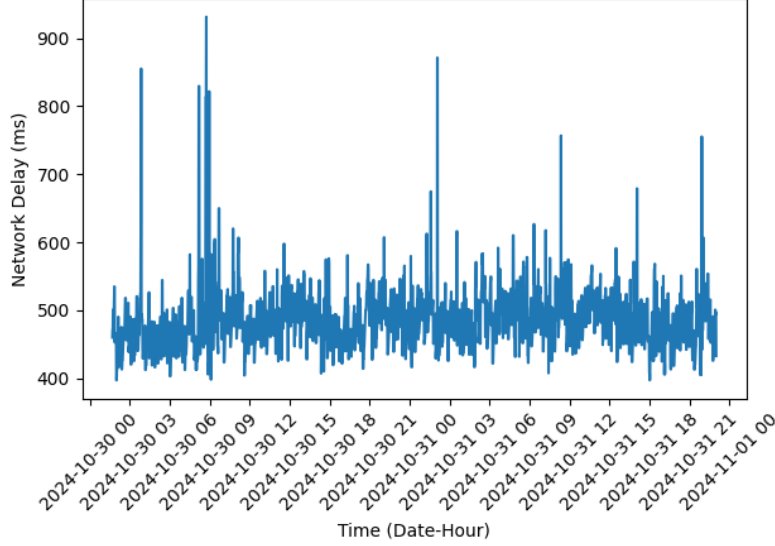


Figure 4.3: System Prefilling Delay Profiling for GPT-4o responses.

For concurrent profiling, while the victim is interacting with the OpenAI server through the GPT4o API, the attacker concurrently makes periodic requests to the LLM using the API (once per minute) to measure the current system prefilling delay and *Time Per Output Token* (TPOT) and estimate the current system load’s impact on token generation times. The attacker also applies a 15-minute sliding window on these measurements, using the median to smooth out fluctuations. Based on the difference in the times measured during offline profiling and online concurrently with the attack, the attacker can apply a correction to the observed victim’s LLM generation times to account for the difference in system load between the profile and attack phases. The token count with concurrent profiling is recovered with the formula, where the system prefilling delay and TPOT are measured from the attacker’s concurrent profile:

$$\text{Recovered Token Count} = \frac{(\text{Victim's Response Time} - \text{System Prefilling Delay})}{\text{Time Per Output Token}} \quad (4.2)$$

For median aggregation, we assume the attacker can observe repeated requests for the same input and send 10 requests per input to the remote OpenAI server. The median response time for an input is taken to recover the output token to mitigate the effect of outliers in the response time due to transient server load peaks. The formula is shown in Equation (4.3). We further demonstrate that combining these two methods yields more precise estimations, leading to improved attack success rates in scenarios that require higher token recovery accuracy, as detailed in Chapter 5 and Chapter 6.

$$\text{Recovered Token Count} = \frac{\text{Median Victim's Response Time Across 10 Repeated Requests}}{\text{Average Time Per Output Token}} \quad (4.3)$$

Transient network congestion is an additional source of noise in timing measurements. However, the network latencies (ping latencies to OpenAI servers are around 100-200ms) make up only a minor fraction of the overall response time (on average 1785ms in our experiments). We observe that OpenAI server load variations dominate the overall measurement noise in the attack phase.

Figure 4.4 shows the linear predictions for the output token count based on the observed response

times (green line), compared with the actual token counts of the victim for the respective response times (blue dots) on GPT-4o. The attacker’s predicted token counts have an NRMSE of 0.465 compared to the ground truth. We observe several outliers that contribute to higher error in the remote GPT-4o setting compared to local servers, likely due to inference-time optimizations such as speculative decoding [63] or transient spikes in server load. Despite this, response time remains strongly correlated with output token count, enabling our attack on OpenAI’s GPT-4o to maintain high ASR, as demonstrated in subsequent sections. For the two previously proposed denoising techniques, concurrent profiling and median aggregation across repeated requests reduce the NRMSEs to 0.324 and 0.316, respectively.

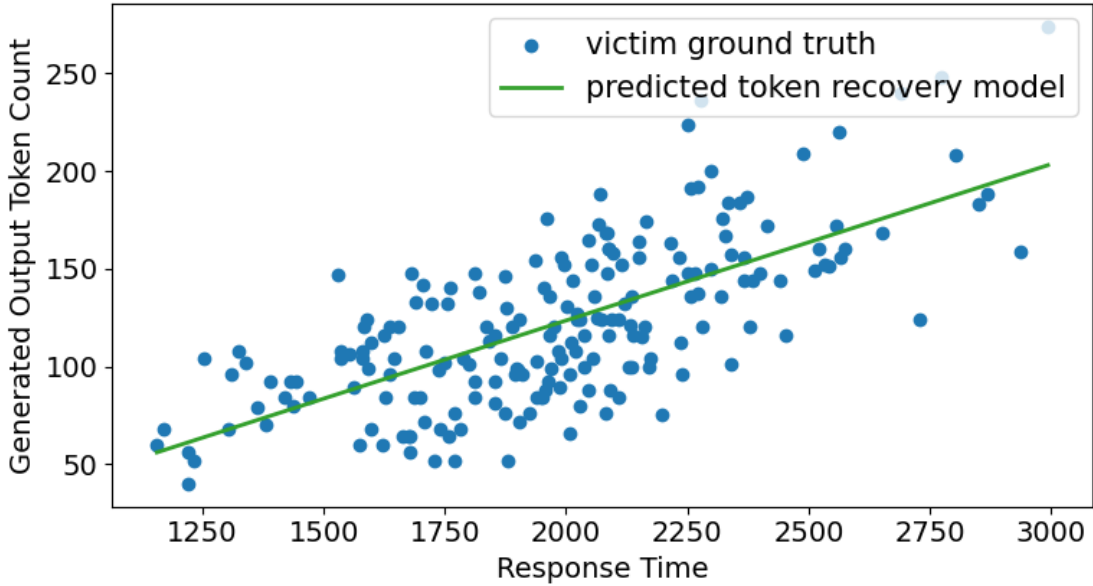


Figure 4.4: Correlation between output token count and response time for GPT-4o model. Attacker’s predictions for token counts (green line) have an NRMSE of 0.465 compared to victim’s actual token counts (blue dots).

4.5 Exploiting Token Count Side Channels

Next, we show how the token count timing side channels can be exploited to leak private user-sensitive information. Specifically, we show how output token count can correlate with private attributes in the user input in classification tasks (Chapter 5) and translation tasks (Chapter 6). Subsequently, leveraging the correlation between generation time and output token counts, we develop end-to-end attacks over the network that infer private user information based on estimations of the output token count from response time.

Chapter 5

Attack on Few-Shot Classification Tasks

5.1 Application Scenario

Context. LLMs are increasingly used for text classification tasks, such as Kasisto’s KAI-GPT for automated mortgage processing [64], and Google Med-PaLM [65] for medical diagnoses. These applications often involve LLMs generating explanations along with the classification output to facilitate error diagnosis and ensure accountability [66, 67]. Additionally, GDPR mandates explanations for AI-based decisions, suggesting that individuals have the right to get clarity about decisions made on their personal data [68]. Consequently, commercial applications are expected to provide explanatory outputs alongside classification results.

Meanwhile, for such domain-specific use cases of LLMs, few-shot prompting [43], which uses a small number of task-specific examples in the input prompt, enables effective task performance without costly fine-tuning. This approach is useful in scenarios with limited or evolving data, delivering performance comparable to fine-tuned models. Research has shown that few-shot examples can introduce biases in the output [44], but can also improve accuracy when combined with appropriate explanations [45–47].

In this section, we explore whether output explanation lengths (i.e., explanation token counts) in classification tasks correlate with output classes, potentially leaking class information. In contrast to prior work [45–47], we investigate whether text classification tasks exhibit inherent biases in output explanation lengths and whether few-shot prompting can unintentionally amplify signals in output token counts, thereby affecting the side channels that leak output classes.

Attack Setup. We target an LLM performing binary text classification in the Predict-then-Explain (P-E) format [45], where the LLM outputs a class label followed by an explanation. The attack leverages the token count timing side channel to infer the predicted class, thus revealing sensitive user information.

5.2 Attack Design

In classification tasks, LLMs may exhibit inherent biases to produce longer explanations for specific classes. When few-shot examples augment this bias, class-dependent differences in output token count become more pronounced, creating a side channel. An adversary can exploit this by observing or estimating the output token count to infer the classification result. To leak information, the attacker must first characterize this behaviour by profiling the task’s output token count distribution for various inputs. Since the input length also affects the output token count, the attacker must also consider this. We perform the attack with unbiased few-shot examples to evaluate the natural potential for leakage in classification tasks and then with biased examples to study the impact of few-shot prompting.

5.3 Attack Implementation

Profiling Phase. In the profiling phase, the attacker uses the output token count (or its estimate using the LLM response time), supplemented by the total input length in bytes, to establish a threshold for distinguishing between two predicted classes. The attacker sends 200 inputs (100 per class) of known length to the LLM for classification and observes the output class and the output token count for each response. Then, the attacker fits an optimal threshold for the output token count that can achieve the highest ASR that best separates the two classes, defined as:

$$\textit{Token-Count Threshold} = \alpha \times \textit{Input Length in Bytes} + \beta \quad (5.1)$$

The optimal threshold is the one that maximizes the attack success rate (ASR) in the profiling phase under the constraint that the prediction precisions for both classes is similar (e.g. 72% vs 68%, instead of 90% vs 50%). We define the optimal ASR as follows, where θ is a hyperparameter to choose how close the ASR between classes should be (we use $\theta = 0.5$ in our experiments when setting the threshold and $\theta = 0$ when reporting the ASR in the evaluation):

$$\textit{Optimal ASR} = \frac{(\textit{Prec.1} + \textit{Prec.2})}{2} - \theta |\textit{Prec.1} - \textit{Prec.2}| \quad (5.2)$$

Attack Phase. The attacker monitors the (1) length in bytes of the encrypted input and the (2) output token count of a user’s classification request and determines whether the result falls above or below the threshold, thereby inferring the predicted class.

5.4 Experiment Setup

Datasets. To get representative classification tasks, we use the Natural Instructions Dataset [69], which consists of over 1500 distinct tasks, their human-authored instructions and inputs. From this dataset, we select 12 binary classification tasks for our experiments, as shown in Table A.1 in the Appendix. While the dataset includes tasks and inputs and outputs, it does not provide explanations for the output, which we need for our few-shot examples. For this reason, we create a synthetic dataset of explanations using these inputs and outputs with GPT-4-turbo [70], which we then use as few-shot examples.

Prompt. The system prompt to the LLM consists of a task description from the Natural Instruction dataset, and an instruction requesting an explanation, and a set of few-shot examples that include predicted labels and explanations in the P-E format. The user prompt then provides the task input, drawn from the “inputs” field of the natural-instruction dataset. The final output generated by the language model will follow the same P-E format as the few-shot examples. As prior work suggests using a balance of examples across different classes for the best performance [44], we use an equal number of few-shot examples per class: one, two, or three few-shot examples per class, which we refer to as one-shot, two-shot, and three-shot, respectively. Figure 5.1 shows the template for our *one-shot* prompt to the LLM.

System Prompt	
Task Description:	<Task Definition> Provide an explanation following the answer. When answering user inputs, follow these examples:
Input:	<Input from Class A>
Answer:	<A>. <Short (One-Sentence) Explanation as Example>
Input:	<Input from Class B>
Answer:	. <Long (Three-Sentence) Explanation as Example>
User Input	
Input:	<Input from Class A>
Assistant Response	
Response for Input:	<A>. <(Expected) Shorter Explanation>

Figure 5.1: Prompt template for our tasks, using the Predict-then-Explain (P-E) format. The system prompt consists of the task description and the input examples for each class (e.g., one-shot example per class). If the few-shot examples have biased explanation lengths (shorter for class A), it is possible the response with output class A also has a shorter explanation.

Experiments. To characterize the biases in the explanation lengths between classes of a given task, we provide each input class either a *short*, one-sentence explanation, or a *long*, three-sentence explanation in the few-shot examples. Thus, we have four categories of examples for the two classes, long-long, and short-short which are *unbiased*, and short-long and long-short that are *biased*. Using unbiased examples, we characterize the inherent bias in the explanation lengths generated for a given task. For biased examples, we categorize them on a per-task level as *augmenting* or *diminishing*, based on whether their bias is the same or opposite direction as the inherent bias in the task. With these examples, we can characterize the diminishing or augmenting effect of biased examples on the output explanation lengths and observe the best/worst-case leakage of output class from token counts for a given task. We use 200 inputs (100 per class) for training profiles and 200 inputs (100 per class) for testing our attack per category.

Models. We evaluate open-source models from the Gemma-2 [71] and Llama-3 [72] family, as well as proprietary models from the GPT-4o [38] family. We select these due to their state-of-the-art

accuracy in classification tasks.

5.5 Profiling Results

5.5.1 Class Profiling for Explanation Length Bias

To characterize the inherent bias in explanation lengths in tasks, and also the best/worst-case biases due to biased few-shot examples, we characterize each of the tasks with unbiased (long-long), and biased examples of the augmenting and diminishing types and measure the output token counts for the two output classes of the task. Figure 5.2 shows the visualization of the output token counts with the Gemma2-9B model for one such task (Task-145) that classifies a pair of arguments into Class A or B, based on whether they support or attack the other’s stance, with three-shot per class.

For the unbiased examples, the inherent bias of Gemma2 causes it to generate longer responses for Class A and shorter ones for Class B. With *augmenting* examples (long-short), the difference is amplified to create a considerable separation in token counts for the two classes. On the contrary, *diminishing* examples (short-long) cause the bias to move in the opposite direction with the classes having a similar distribution of token counts. This validates the fact that LLMs have natural explanation length biases based on output classes, and these can get augmented or diminished based on the biases in the example explanation lengths.

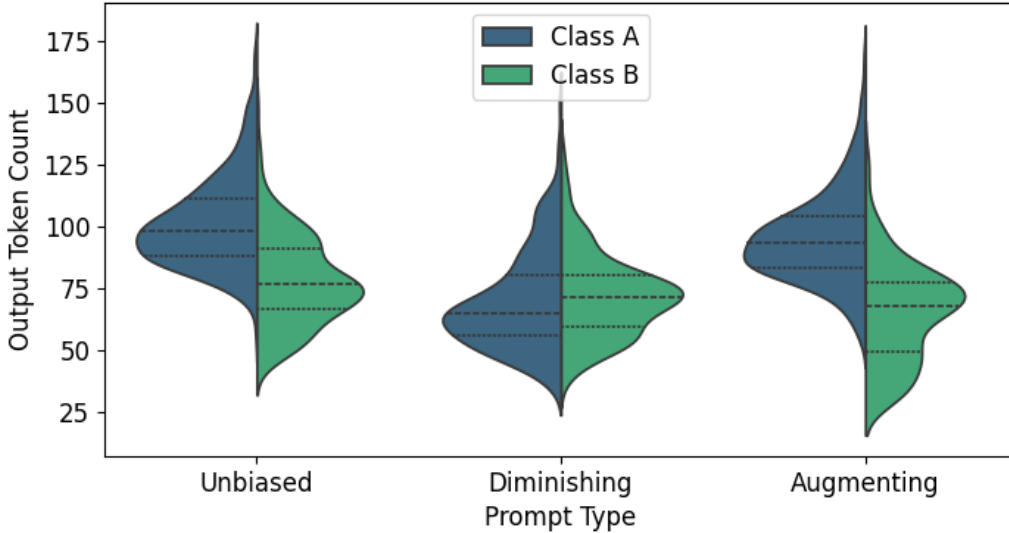


Figure 5.2: Output Token Distribution for Task-145, with Unbiased, Augmenting and Diminishing examples used as few-shot examples (three-shot per class). The Unbiased is chosen as “Long-Long”.

Using Equation (5.1) and Equation (5.2), we derive the thresholds for each of the tasks with each model and few-shot example category, which enables our attack.

5.5.2 Attack Success Rate with Output Token Count

Figure 5.3 shows the ASRs across all 12 tasks for the Gemma2-9B model with 3-shot examples per class. With the baseline unbiased examples, we see that the average ASR reaches 63.1% (short-short) and 66.7% (long-long), with some tasks having ASR higher than 75%. This is much higher than

the ASR with random guesses (50%), indicating that the output token counts are naturally biased in tasks even without any biased examples. When the task is prompted with *augmenting* few-shot examples, the distribution shifts even more towards the inherent bias, leading to an average 15% increase in the attack success rate. In the *diminishing* direction, the results are mixed. Either the explanation exhibits a less significant difference in the opposite direction, or the explanation bias is in the same direction as inherent bias but to a smaller degree. The diminishing examples result in a best-case ASR of 75% (Task 227) and an average ASR of 55%.

Task-679 is an outlier where all of the categories (unbiased or biased) have low ASR (around 50%). This is because the output token count for this task does not have a strong bias for either of the classes, making our attack ineffective. With biased examples, it tends to shift a subset of explanations to a disproportionately long length, making the current threshold less stable. Overall, our side-channel attack is quite successful in leaking output class based on output token counts.

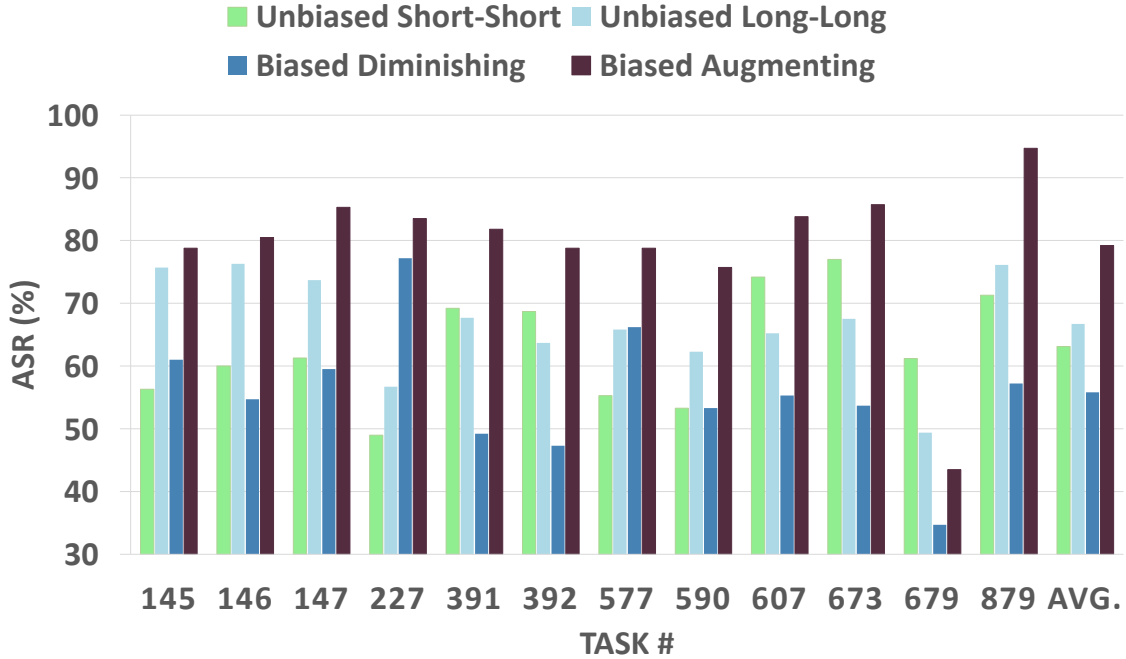


Figure 5.3: ASR (%) across all tasks for Gemma2-9B, with unbiased and biased examples with 3-shot examples per class.

Table 5.1: ASR (%) of Augmenting class prompts for different model architectures and sizes

	Gemma2-2B	Gemma2-9B	Gemma2-27B	Llama3.1-8B	Llama3.2-3B	GPT-4o	GPT-4o-mini
391	79.2	81.8	78.3	83.3	70.5	80.3	92.5
227	70.5	83.5	72.2	83.8	73.8	91.0	94.2
392	67.8	78.8	83.8	86.0	69.7	82.3	94.2
590	70.3	75.7	63.5	69.7	66.2	87.8	87.0
879	89.3	94.7	82.1	93.5	81.5	93.3	97.7
Avg.	75.4	82.9	76.0	83.3	72.3	86.9	93.1

5.6 Timing Attack on Local LLM Servers

In this section, we use the response time from a local LLM server as the proxy for the output token count for our attack. We conducted our evaluation using open-source models from the Gemma2 family. We selected the Gemma2-9b-IT model, which was executed on a local machine with the same spec as in Section 4.2. The timing here is taken directly on the server side. The results are presented in Table 5.2. Our analysis shows that the ASR drop in all tasks is relatively minor, with an average variation of only 1.7% in ASR. This finding indicates that the timing signal closely resembles the output token count, making the timing side channel a viable means of leaking sensitive attributes.

Table 5.2: ASR (%) of Gemma-9B-IT with Augmenting class prompts with timing information

Task #	One-Shot	Two-Shot	Three-Shot
145	57.8(-8.4)	64.2(-12.3)	69.0(-9.8)
146	63.5(-3.3)	72.2(-8.0)	72.3(-8.2)
147	86.3(-0.2)	85.2(-0.6)	85.5(+0.2)
227	73.0(+6.2)	79.5(-0.5)	79.0(-4.5)
391	77.0(-0.7)	78.8(-1.2)	81.0(-0.8)
392	74.0(+0.0)	78.7(-0.3)	79.0(+0.2)
577	69.4(-0.1)	77.9(+1.3)	79.7(+0.9)
590	71.2(+1.5)	73.2(-0.1)	78.3(+2.6)
607	75.9(-0.5)	83.2(+2.8)	84.9(-0.2)
673	80.8(+0.0)	87.3(-0.5)	85.7(+0.0)
679	58.6(-0.2)	44.1(-1.2)	43.5(+0.0)
879	86.3(+0.3)	92.5(+0.7)	93.9(-0.8)
Average	72.8(-0.5)	76.4(-1.7)	77.6(-1.7)

5.7 Timing Attack on GPT-4o on Remote OpenAI Servers

As discussed in Section 4.4, the proprietary GPT-4o exhibits more variations in delay than our open-source models. We thus study the effectiveness of three different profiling methods. The first method measures the time between sending a request to the OpenAI API and receiving a response, assuming a linear correlation with output token count and ignoring other sources of timing variation. This naïve method serves as the baseline for these experiments. The second method utilizes concurrent profiling discussed in Section 4.4 to monitor the variations in the execution time. This measurement helps us estimate network and system delays concurrently with the attack, as well as the Time per Output Token (TPOT). Measuring network and system delay concurrently enables the attacker to account for varying server loads throughout the day. By combining these two pieces of information, we can estimate the number of output tokens for a request made at a similar time during the concurrent profiling session using Equation (4.2). The third and final method assumes access to 10 repeated requests of the same input and aggregates the median timing delay as a proxy for the output token count (Equation (4.3)). For the class profile to be used on all tests, we take 10 profiles across 5 days and combine them into one profile. The class profile is adapted for each method and applied to both training and testing phases.

The ASR across 10 profiles for the first two network profiling techniques is presented in Fig-

ure 5.4. Concurrent profiling achieves a higher average ASR and exhibits greater stability across days. Notably, it yields an 2.1% ASR increase on average and higher stability with 6.2% higher ASR than the baseline method on Profile 5, indicating improved robustness to server load variability. When the server load remains relatively stable, concurrent profiling performs similarly to the baseline methods. Using concurrent profiling, the average ASR on GPT-4o is only 3.8% lower than that of open-source models.

As discussed in Equation (5.2), beyond maximizing average precision across both classes, it is also important to maintain a balanced precision between them. Under timing drift, the average precision may remain relatively stable while the per-class precisions become imbalanced. To capture this, we evaluate an additional metric: the harmonic mean of the two class precisions (5.3). For the static method, the mean harmonic precision is 66.4%, whereas the concurrent profiling method achieves 73.0%, demonstrating significantly greater robustness to server load variations and reduced sensitivity to threshold drift.

$$\text{Harmonic Mean ASR} = \frac{2 \cdot \text{Precision1} \cdot \text{Precision2}}{\text{Precision1} + \text{Precision2}} \quad (5.3)$$

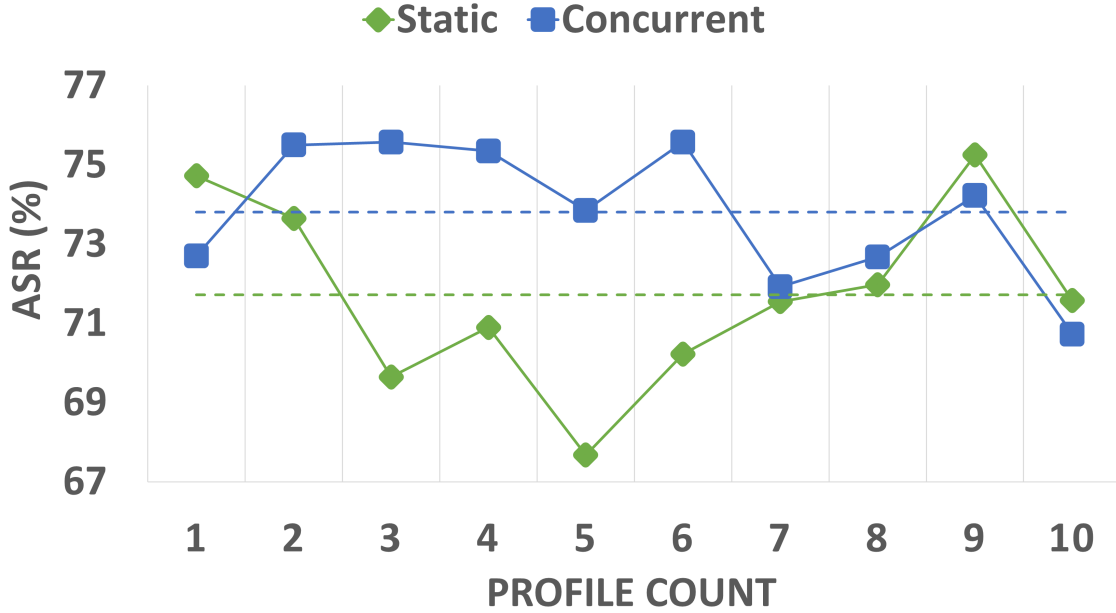


Figure 5.4: Average ASR (%) across tasks for GPT-4o on different days

Additionally, we evaluate a setting in which the attacker has access to all 10 test profiles and uses the median elapsed time for both training and testing phases. Under this assumption, the attack achieves a significantly higher ASR of 84.9%. When combined with concurrent profiling, the ASR further improves to 85.3%, representing the highest performance observed across all evaluated methods.

5.8 Additional Results

We conduct additional experiments assuming direct access to output token counts to analyze how it may work in real-world applications with sensitive attributes and factors influencing explanation length bias across different models.

5.8.1 Dataset Setup and Results for Motivation Examples in Classification Attack

For the loan application approval scenario, we utilize the Loan Approval Classification Dataset [73], which comprises a data spreadsheet containing synthetic users’ personal information and the approval status of their applications. We use GPT-4.5-preview to generate descriptions from this data in natural language, utilizing various sentence structures. This description is fed into the testing model along with a task description, prompting the model to determine whether the loan application should be approved or not.

For the diabetes diagnosis, we take the SymptomsXDiagnosis dataset [74], which includes three fields: disease type, symptoms, and diagnosis. The task is to determine whether the symptom description matches the typical symptoms of diabetes, and the target class is ‘yes’ for likely to be diabetes and ‘no’ for otherwise. We use the diagnosis as the explanation and ask GPT-4.5-preview to rewrite it into explanations consisting of 2 or 5 sentences, respectively, for short and long responses. The 100 diabetes disease samples are used as the positive class and supplemented with 15 extra synthetic samples to ensure sufficient sample sizes for few-shot examples and testing inputs. Some question-style symptom descriptions have been revised to include symptom descriptions of 2 to 3 sentences each. The negative class examples are sampled from 20 other diseases.

The results are presented below:

Table 5.3: ASR (%) for two motivation examples, with Augmenting 3-shot examples and direct access to output token count.

Task Name	Gemma2-9B	Llama3-8B	GPT-4o
loan-approval	80.5	65.8	82.6
diabetes-diagnosis	75.9	74.8	81.0

5.8.2 Varying Number of Few Shot Examples

Figure 5.5 shows the ASR as the number of few-shot examples per class is varied, for the *augmenting* examples, for all the tasks with Gemma2-9B. On average, the ASR increases significantly from one-shot (72.9%) to two-shot (78.1%), and then shows a moderate increase to three-shot (79.2%). This shows that the biases in output token counts for most tasks get accentuated when more biased examples are added, although this effect has diminishing returns.

5.8.3 Alternative Model Architectures and Sizes

To validate that the biases persist across different LLMs and whether there is a correlation with model size, we test our attacks on several state-of-the-art LLMs, including Gemma-2, LLaMa-3, and GPT-4o models, with varying model sizes. Due to the large run times for these experiments, we

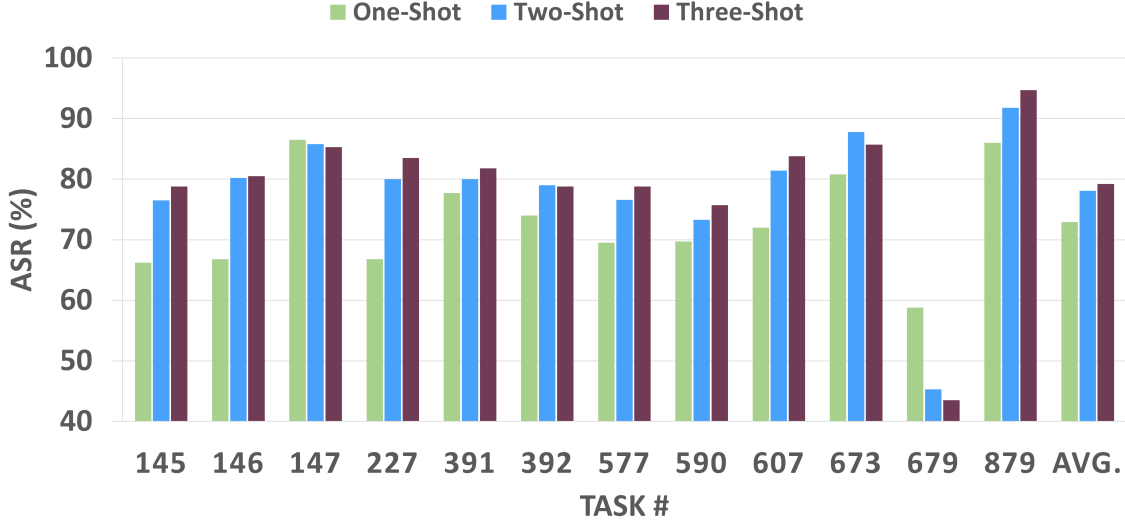


Figure 5.5: ASR (%) as the number of few-shot examples is varied from 1-shot to 2-shot to 3-shot, for Gemma2-9B, with augmenting examples.

only tested a subset of 5 representative tasks across all models with the *augmenting* examples and 3-shot prompts. Table 5.1 shows the results.

We observe that the GPT-4o family has the highest ASR, with GPT-4o having an average ASR of 86.9% and GPT-4o mini (93.1%); this is due to strong bias in the output token counts. In comparison, LLaMa-3.1 and 3.2 have average ASR of 83.3% and 72.3%, respectively, and Gemma-2 models have average ASR between 75.4% and 82.9%. There is no strong correlation with model size. While GPT-4o-mini shows more bias than its larger counterpart GPT-4o, Llama-3.1/3.2 shows the opposite behaviour with less bias (lower ASR) for the smaller model size, and Gemma2 has similar lower ASR for the smallest (2B) and the largest model (27B) sizes.

5.8.4 Approximate Prompt

To show the attack is robust to minor variations in the attacker’s knowledge of task descriptions, we run the attack after rewriting the task descriptions with GPT-4.5-preview to get alternative variants of the task prompts and few-shot examples which retain the same semantic meanings. We show the results in Table 5.4. The ASR variations are within 1.5% across all three models, demonstrating that the attack is mainly correlated with the task semantics itself.

Table 5.4: ASR (%) for approximate prompts, with Augmenting 3-shot examples (difference of ASR from the original prompt in the training set across all model predictions in brackets).

Task #	Gemma2-9B	Llama3-8B	GPT-4o
227	78.8 (+2.1)	84.2 (-4.2)	86.3 (+0.7)
590	76.8 (+2.2)	78.2 (-2.7)	79.5 (-1.3)
673	86.7 (-0.2)	82.7 (+0.8)	84.7 (-2.4)
879	92.7 (-0.7)	81.9 (+0.2)	84.8 (-3.1)
Average	83.8 (+0.9)	81.8 (-1.5)	83.8 (-1.4)

5.8.5 Ablation studies

Model Temperature. A higher model temperature introduces randomness in generated outputs and may add noise in the output token distribution that impacts our attack. Our default experiments were done with a temperature of 0 (or 0.0001 when a zero temperature is not supported by the model) for reproducibility. To test the impact of temperature, we vary the temperature from 0 to 0.3 to 0.7 and evaluate the ASR (with augmenting 3-shot examples) for the GPT-4o and Gemma-2 models. For GPT-4o, the ASR drops from 82.9% to 82.1%, to 80.8%, whereas for Gemma-2, it varies from 86.9% to 87.1% to 87.8%, as the temperature goes from 0 to 0.3 to 0.7. Thus, our attack is highly resilient even under relatively high temperatures for classification tasks. Table 5.5 provides the ASR results across all tasks.

Table 5.5: ASR (%) as the temperature varies, for the augmenting 3-shot examples. Gemma stands for Gemma2-9B-IT model.

Task #	Temp=0		Temp=0.3		Temp=0.7	
	Gemma	GPT	Gemma	GPT	Gemma	GPT
227	83.5	91.0	81.5	88.3	77.0	89.3
391	81.8	80.3	80.6	84.6	79.2	85.8
392	78.8	82.3	78.7	85.0	77.3	86.7
590	75.7	87.8	72.2	86.0	74.7	84.5
879	94.7	93.3	97.3	91.7	95.6	92.5
Average	82.9	86.9	82.1	87.1	80.8	87.8

Correct Model Predictions. Although model accuracy is not our primary focus, we wish to examine whether correct model class predictions have a correlation with the length of generated output explanation. To study this, we filter the results to only include cases where the predicted target matches the ground truth. With this filtering, the average ASR increases slightly by 1%. However, this change is not substantial enough to suggest any correlation between correct predictions and bias in output lengths. Table 5.6 shows task-wise ASRs for Gemma2-9B and GPT-4o.

Table 5.6: ASR (%) only for results with only correct model predictions, with Augmenting 3-shot examples (difference with ASR across all model predictions in brackets).

Task #	Gemma2-9B	GPT-4o
145	80.8 (+2.0)	91.7 (+1.2)
146	84.9 (+4.4)	96.1 (+2.8)
147	85.8 (+0.5)	95.1 (+3.4)
227	82.0 (-1.5)	89.9 (-1.1)
391	81.3 (-0.5)	82.7 (+2.4)
392	80.1 (+1.3)	83.0 (+0.7)
577	86.9 (+8.1)	93.8 (+4.1)
590	75.1 (-0.6)	87.6 (-0.2)
607	85.6 (+1.8)	88.4 (-1.8)
673	86.3 (+0.6)	90.6 (+0.4)
679	38.6 (-4.9)	82.7 (+1.0)
879	94.6 (-0.1)	94.9 (+1.6)
Average	80.2 (+0.9)	89.7 (+1.2)

Chapter 6

Attack on Translation Workloads

6.1 Application Scenario

Context. In recent years, LLMs have become the premier choice for machine translation, powering services like Google Cloud’s adaptive translation [13] and DeepL’s next-generation translator [14]. Given the potential privacy implications discussed in Chapter 1, translation service providers must protect users’ language preferences from adversarial inference. **Attack Setup.** We assume the user sends multiple translation requests using a fixed source-to-target language pair. Given the known source language, the attacker seeks to infer the unknown target language. By monitoring LLM request-response timings and input/output lengths, the attacker attempts to recover the target language. We define the Attack Success Rate (ASR) of this attack as the precision of the attacker’s prediction about the target language, using 50 translation requests from the user.

6.2 Attack Design

Overview. LLMs use subword tokenizers, commonly based on Byte Pair Encoding (BPE), to construct vocabularies guided by token frequency. Biases in training data cause token vocabularies to favour high-resource languages, resulting in more tokens for similar content in lower-resource languages [22, 23]. Language morphology differences, such as word length and the use of prefixes and suffixes, further impact vocabulary construction and token density. We exploit these factors to construct a side channel, enabling attackers to infer the target language via token density and output-input byte ratios.

Figure 6.1 shows an example of an attacker profiling the characteristics of three different languages (French, Chinese, Spanish) by requesting the input “*Let’s translate this text*” to be translated into each of these languages. The attacker measures the number of output tokens and the lengths of the input request and output response in natural language. For the same input, the output in each language has a different *token density*¹ (length of output in bytes / # output tokens), due to the tokenizer’s vocabulary differences. Moreover, due to the morphological differences, these responses have different *output-input byte ratios*: sentences with the same semantic meaning can have different

¹Note that we use output token density, instead of just # output tokens, since we seek a metric that is independent of the output length, as the inputs and outputs for a given user request can be arbitrarily long.

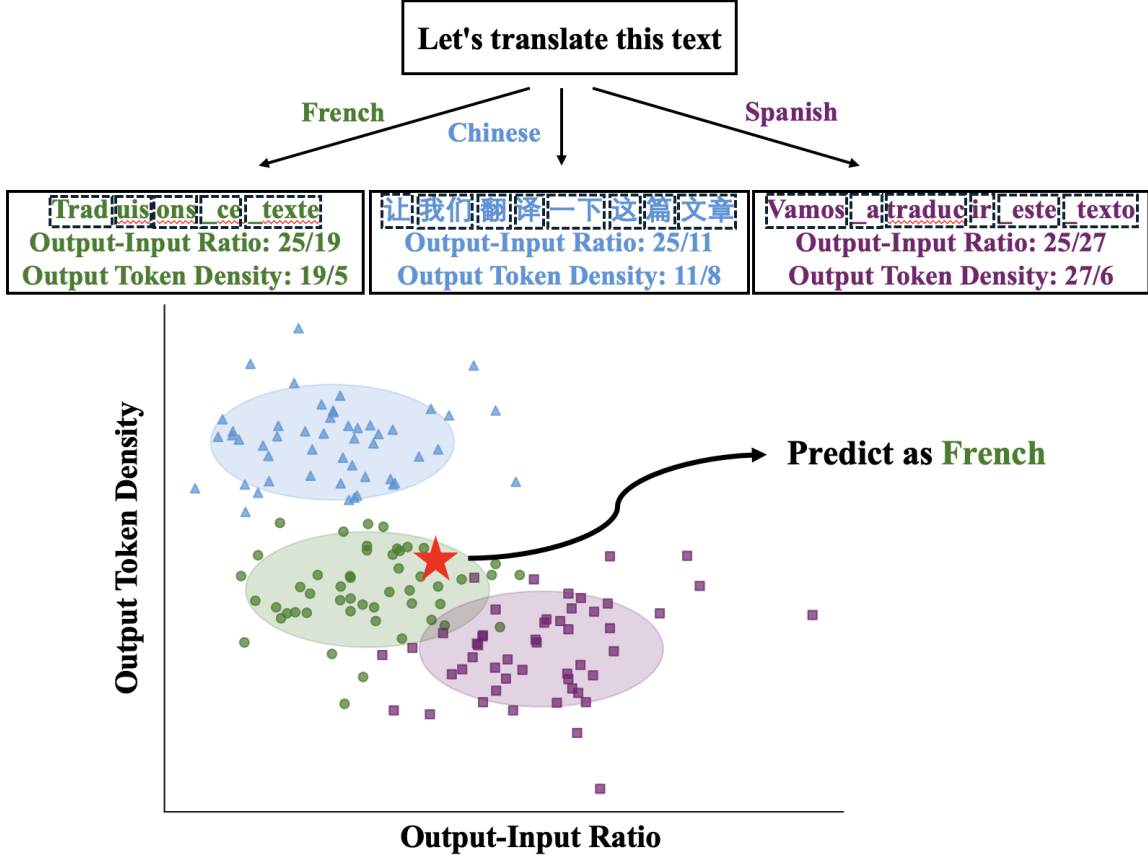


Figure 6.1: Overview of Attack on Translation. A user translates text in a source language (e.g., English) to any target language; the goal of the attack is to leak this target language. By profiling output token density and output/input bytes ratio and observing these for a given translation, the attacker can leak the user’s target language.

byte lengths in different languages.

The attacker can thus develop a 2D class profile for different target languages using these two metrics: (1) *output token density* and (2) *output-input byte ratio*, using a large number of inputs offline. Subsequently, by monitoring a user’s translation request and response and calculating the above two metrics, the attacker can map these to a point in this 2D profile and recover the user’s unknown target language with high ASR.

6.3 Attack Implementation

Profiling Phase. In the profile phase, the attacker sends 1000 requests per language to the LLM to be translated and measures the *Output Token Density* and *Output-Input Ratio* for each request-response, formalized as follows. For a request and response pair, with input length in bytes as L_{input} , and the output length in bytes as L_{output} , the (1) *output token density* and (2) *output-input ratio* are calculated as:

$$\text{Output Token Density} = \frac{L_{\text{output}}}{\# \text{ of Generated Tokens}} \quad (6.1)$$

$$\text{Output-Input Ratio} = \frac{L_{\text{output}}}{L_{\text{input}}} \quad (6.2)$$

Using the profiling data, the attacker learns two Gaussian distributions for the output-input length ratio and token density as the *class profile*. These two distributions are used to construct a two-dimensional Gaussian Mixture Model (GMM) [75] for each target language (given a model and source language) that describes its characteristics (see Figure 6.2). We use this profiled 2D GMM to launch our attack.

Attack Phase. In the attack phase, we assume the user sends multiple translation requests (1 to 50) to the LLM for the same source-target language pair. The attacker gathers the input and output encrypted text lengths and the generated output token count across these requests and constructs GMMs based on this data. We use the Bhattacharyya Distance [76] to map this GMM to the most similar GMM in the learned *class profile* to leak the target language.

6.4 Experiment Setup

Models. To evaluate our attack, we prioritize multilingual models that consider language parity [22] and achieve high translation performance across a wide range of languages, as measured by BLEU scores. Based on these constraints, we test on the following models: M2M100 [52], MBart50 [77], and Tower [78]. M2M100, from Facebook, is specialized for direct many-to-many multilingual translation. It uses a single shared vocabulary across all languages, built with a SentencePiece tokenizer. The model allows direct translation between any pairs of languages from among 100 languages. MBart50 supports up to 50 languages and similarly uses a shared vocabulary created with SentencePiece. However, it is trained on monolingual data in 50 languages, followed by multilingual fine-tuning with parallel data. Tower is a variant of LLaMA-2 that enhances the multilingual capability of the vanilla LLaMA-2 model for 10 selected languages. Its vocabulary follows LLaMA-2 without extension. In comparison to these models, English-centric models like OpenAI’s GPT exhibit even larger disparity in the tokenizer’s vocabulary across different languages [22], hence, we expect our attacks to be effective on them as well.

Datasets. For our dataset, we use the Flores dataset [79], which provides translations of the same sentence across various languages. This dataset facilitates the evaluation of translation quality using ground truth results available for numerous languages. Moreover, it supports testing across a range of different {source language, target language} pairs, ensuring that our results are generalizable. The dataset is divided into training and testing sets, and we maintain the same split to ensure our results are reproducible. We profile output token density and output-to-input length ratio distributions from the training set with 1000 inputs per target language and attempt to infer the unknown target language in the testing set with 50 samples. Additionally, we use the ground truth translations to evaluate translation quality using BLEU scores for each target language in Table 6.1.

Languages. For testing our attack, we selected target languages with the highest speaking populations, while ensuring sufficient diversity across different language families. The 13 languages we use for the target language to test our attack are shown in Table 6.2. Note that Tower only supports 10 out of these 13 languages, and we exclude the unsupported languages (Hindi, Arabic, Japanese) for this model. For M2M100 and Tower, we perform the testing using English, French, Spanish, and Russian as the source languages. For MBart50, which needs fine-tuning for a particular source

language, we only use English as the source language. We ensure all the models perform translation with sufficiently high accuracy, based on the BLEU score. Table 6.1 shows the translation performance of three models—M2M100, MBart50, and Tower—across various languages, measured using BLEU scores. Overall, all models achieve sufficiently high accuracy, with the Tower model generally outperforming the others, especially in languages like French, Portuguese, and Dutch. High scores in widely spoken languages reflect effective training and robust multilingual capabilities, while reasonable performance in less-resourced languages like Chinese and Korean highlights the models’ ability to generalize. These models are thus well-suited for diverse multilingual translation tasks.

Table 6.1: Translation performance (BLEU scores) for different models, M2M100, MBart50, Tower.

	Chinese	French	Hindi	Spanish	Arabic	Portuguese	Russian	German	Japanese	Korean	Italian	Dutch
M2M100	19.0	44.7	26.6	25.4	13.1	45.6	28.1	35.4	28.5	21.9	26.8	25.0
MBart50	19.0	42.6	22.5	21.4	17.1	33.8	24.8	34.1	28.0	19.6	24.4	22.6
Tower	21.4	49.3	-	27.2	-	49.8	30.4	37.3	-	24.4	29.8	27.3

Table 6.2: Target Languages Used to Test our Attack, Grouped by Language Family

Language Family	Languages
Indo-European (Romance)	French, Spanish, Portuguese, Italian
Indo-European (Indo-Aryan)	Hindi
Indo-European (Germanic)	German, Dutch, English
Indo-European (Slavic)	Russian
Sino-Tibetan (Sinitic)	Chinese
Afro-Asiatic (Semitic)	Arabic
Japonic	Japanese
Koreanic	Korean

6.5 Profiling Results

6.5.1 Class Profiling

Figure 6.2 shows a profile that we developed using the Tower [78] model, with the English source language and 9 different target languages, listed in Table 6.2 (excluding 3 languages that the model doesn’t support). We profile 1000 inputs per language, repeating the same inputs for each language from the training split of the Flores-200 [79] dataset. As shown in Figure 6.2, both the Output Token Density and Output-Input Bytes Ratio play a distinct role in helping cluster data points of any given language. Particularly, resource-poor languages with low token density (low bytes per token), such as Chinese and Korean, and those with distinctive morphology and Output/Input Bytes like Russian are easily identifiable in this profile. Other languages can be identified based on the combination of the two metrics. We construct similar profiles for each model across all source languages we test.

6.5.2 Attack Success Rate with Output Token Count

Figure 6.3 shows the ASR for our attack correctly guessing the target language on three different models, using English as the source language. Overall our attack is highly effective, with the average ASR across all languages reaching 82.5% for Tower, 77.3% for M2M100 and 80.6% for MBart50,

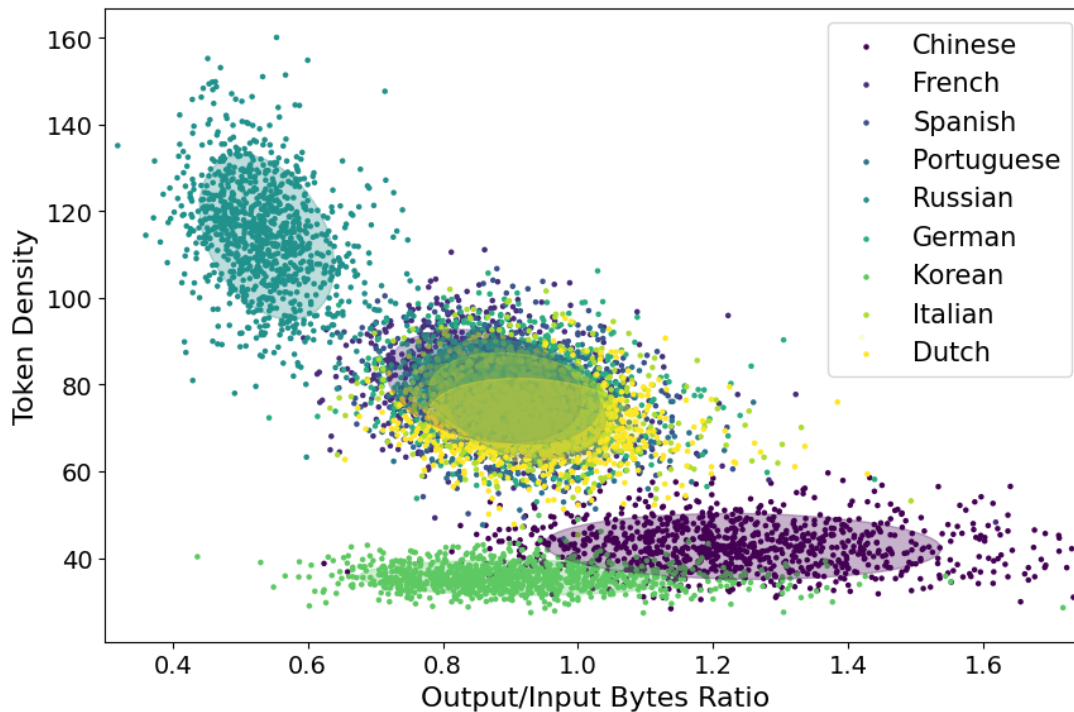


Figure 6.2: Profile using a Gaussian Mixture Model on a 2D decision space for the Tower model with the source language as English.

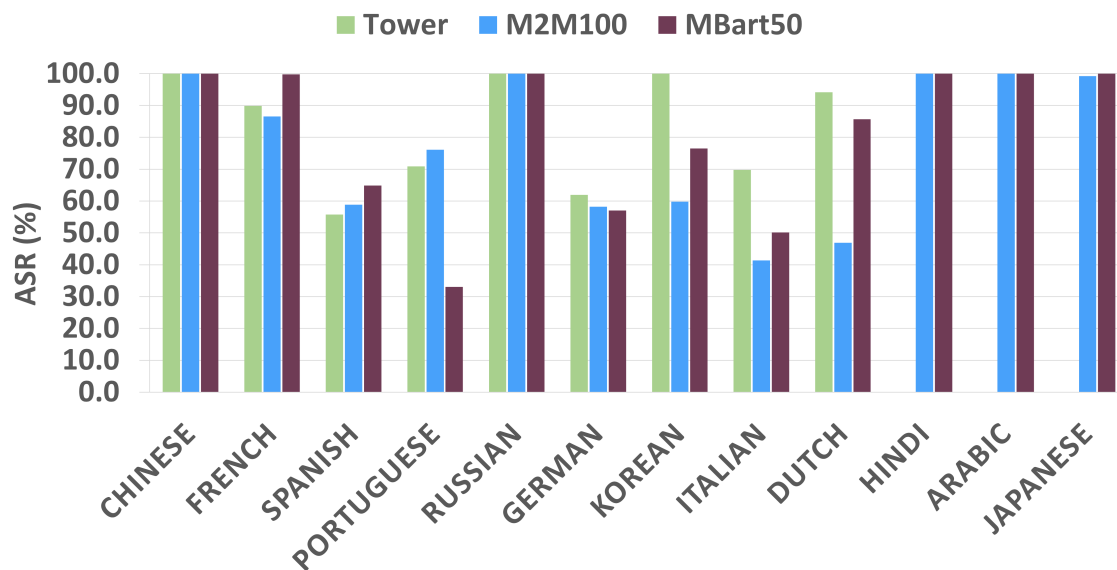


Figure 6.3: Attack success rates (ASR) for the attack leaking the output languages for different translation models, using English as the source language.

respectively. Languages with more distinct morphology from different families, including Chinese, Russian and Korean, show 100% ASR for the Tower model. This is due to major differences in the byte encoding scheme, vocabulary sizes, and information density embedded in similar text lengths. Four languages from the Indo-European (Romance) family achieve the lowest ASRs, where one

language is more likely to be misclassified as another in the same family.

6.6 Timing Attack on Local LLM Servers

In this section, we consider a local LLM server operating in non-streaming mode. Under this setting, the attacker learns the *class profile* using response time as a proxy for the output token count.

As the intrinsic nature of autoregression suggests, an approximate linear relationship exists between the number of generated tokens and the execution time. We have:

$$\# \text{ of Generated Tokens} \propto t \quad (6.3)$$

Since we don't need the exact number for token density but rather their relative relationship, we can effectively recover this relationship by approximating it with the execution time. Therefore, the token density calculation can be expressed as:

$$\text{Token Density} = \frac{L_{\text{output}}}{\# \text{ of Generated Tokens}} \propto \frac{L_{\text{output}}}{t} \quad (6.4)$$

where L_{output} represents the byte length of the output.

Table 6.3: Language prediction ASR in the network scenario with timing information for 50 samples (Difference between timing and output token ASR shown in brackets)

Language	Tower	M2M100	MBart50
Chinese	100.0(+0.0)	100.0(+0.0)	100.0(+0.0)
French	90.7(+0.8)	83.4(-3.1)	100.0(+0.2)
Spanish	51.8(-4.0)	63.5(+4.6)	67.8(+2.9)
Portuguese	72.8(+1.9)	75.3(-0.8)	34.0(+1.0)
Russian	100.0(+0.0)	100.0(+0.0)	100.0(+0.0)
German	63.1(+1.1)	44.0(-14.2)	58.8(+1.8)
Korean	100.0(+0.0)	60.4(+0.7)	77.6(+1.2)
Italian	73.4(+3.6)	38.7(-2.7)	50.3(+0.2)
Dutch	97.0(+2.9)	45.4(-1.5)	87.7(+2.0)
Hindi	-	100.0(+0.0)	100.0(+0.0)
Arabic	-	100.0(+0.0)	100.0(+0.0)
Japanese	-	98.5(-0.7)	99.7(-0.3)
Average	83.2(+0.7)	75.8(-1.5)	81.3(+0.7)

We choose the same models as used in our previous evaluation, including M2M100, Tower, and MBart50. We implement the client side on an AWS t3.large instance located in Ohio, which sends translation requests. The server hosts LLM on a g6.xlarge instance with NVIDIA L4 Tensor Core GPUs, located in Oregon. The client first sends the target language and the sentence to be translated to the server. After performing the translation, the server responds with the translated text in plaintext. We estimate the round trip time using a ping request and subtract this time from the measured latency to estimate the execution time of the LLM. We compare the ASR of recovering the target language using timing versus having access to the actual output token count in Table 6.3, where a positive value in the brackets means that recovery accuracy with timing is actually *higher* than with the actual output token count. We attribute slight differences (i.e. less

than 5%) to measurement error—the only one with noticeable performance change is with German on the M2M100 model where recovery with timing experienced a 14.2% ASR drop.

6.7 Timing Attack on GPT-4o on Remote OpenAI Servers

In this section, we repeat the experiment with GPT-4o model on the translation tasks, translating from English into 12 different target languages. Each input in the dataset is evaluated over 10 runs across a span of five days. The baseline method uses the response time as direct swap in place of output token count in the same way as in the local server scenario and achieves an average of 77.2% ASR. We further test two more methods: median aggregation and its variation combined with concurrent profiling. With the assumption that the attacker can monitor the same request 10 times for each input in the dataset, taking the median response time across samples improves the ASR by 2% to 79.1%. Combining this with concurrent profiling further increases the ASR to 81.6%, matching the ASR of the previous three open-source models running on local servers. The detailed ASR breakdown for each language is presented in Figure 6.4.

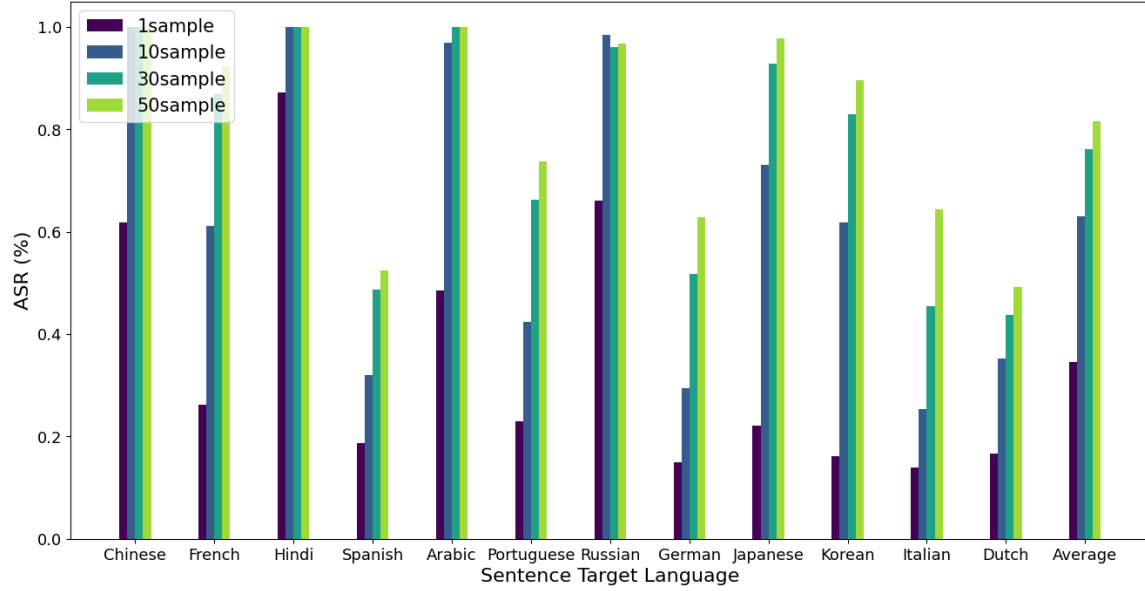


Figure 6.4: ASR (%) for recovering target language in GPT-4o using response time averaged across ten repeated requests with concurrent profiling.

6.8 Additional Results

6.8.1 Varying Attacker Samples

In addition, we analyze the impact of varying the number of user requests the attacker samples to perform the prediction. While our default attack uses 50 requests from the user for the prediction, as the number of samples (user requests) used for prediction increases from 1 to 10 to 30 to 50, the ASR also increases on average from 48.0% to 66.3% to 76.5% to 82.5% respectively, as shown in Figure 6.5.

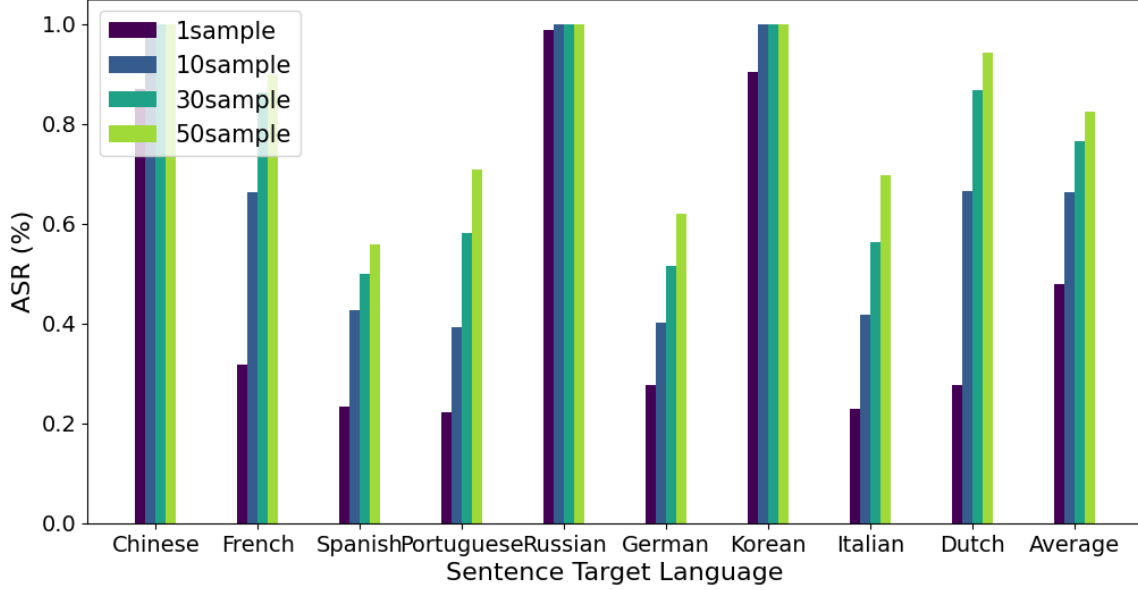


Figure 6.5: Attack success rate (ASR) for the translation attack on the Tower model as the number of sampled requests from the user increases from 1 to 10 to 30 to 50 samples (default = 50).

6.8.2 Alternative Source Languages

While we perform our attack using English as the default source language, our attack generalizes to other source languages as well. For the Tower model, our ASR remains high across other source languages such as French (92%), Spanish (93.3%) and Russian (77.7%). Similar for the M2M100 model, our ASR is sufficiently high across other source languages such as French (81.3%), Spanish (80.4%), Russian (75%). Table 6.4a and Table 6.4b provide the ASRs across all target languages for each of these source languages for Tower and M2M100.

Table 6.4: ASR (%) for recovering target language with source languages other than English for Tower and M2M100 model.

(a) ASR (%) for Tower

Source →	French	Spanish	Russian
English	100.0	100.0	100.0
Chinese	100.0	100.0	100.0
French	-	89.6	69.3
Spanish	77.7	-	36.7
Portuguese	76.2	87.7	76.7
Russian	100.0	100.0	-
German	92.7	86.2	42.9
Korean	100.0	100.0	100.0
Italian	85.5	79.3	79.3
Dutch	95.9	97.3	94.4
Average	92.0	93.3	77.7

(b) ASR (%) for M2M100

Source →	French	Spanish	Russian
English	97.5	100.0	99.8
Chinese	100.0	100.0	100.0
French	-	87.3	68.6
Spanish	83.5	-	55.3
Portuguese	67.2	65.2	62.9
Russian	100.0	100.0	-
German	68.9	60.6	59.1
Korean	64.9	70.8	55.6
Italian	48.7	37.3	36.4
Dutch	54.2	49.5	67.0
Hindi	100.0	100.0	100.0
Arabic	100.0	100.0	100.0
Japanese	91.3	94.5	98.6
Average	81.3	80.4	75.3

6.8.3 Alternative Test Dataset

We also demonstrate that the attacker does not need to specifically profile on the same dataset that the user uses for translation. For instance, we perform the same attack, where the user’s translations (test dataset) are selected from the EuroParl [80] translation dataset. The profiling still uses the Gaussian Mixture Model (GMM) trained with the Flores-200 training dataset. We show that even when the test set is the EuroParl dataset, the attack achieves comparable ASR (79.1%) for the Tower model, as the ASR on the Flores-200 testing set (82.5%). The ASR results across languages are provided in Figure 6.6.

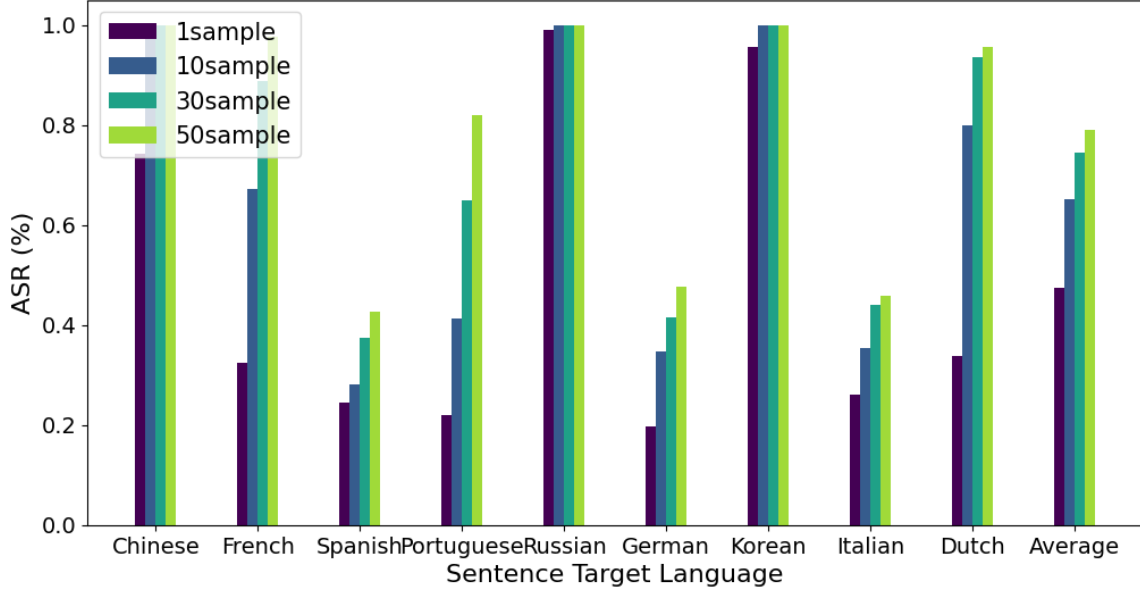


Figure 6.6: Attack success rate (ASR) for the translation attack on Tower model using EuroParl dataset as test set (retaining Flores as training set).

6.8.4 Ablation Study

In addition, we conducted an ablation study by including only one of the two attack metrics, i.e., the output token density or the output-input length ratio, and compared this against an attack using both these metrics (combined channel). Figure 6.7 compares these metrics for the Tower model. The results show that token density contains slightly more information than the output-input length ratio; however, both attributes contribute towards increasing the overall ASR in the combined channel. For instance, token density and output-input length ratio alone only achieve an average ASR of around 70%, whereas together, they achieve an ASR of 82.5%. Our ablation studies for the other models, such as M2M100 (Figure 6.8) and MBart50 (Figure 6.9), are shown below and provide similar insights.

These results show that our attack can predict the language of LLM outputs even when a user is performing non-translation tasks simply by analyzing the token density. Since the token-density channel is a fundamental property of the LLM and such attacks achieve high average ASR (70%), this potentially puts a user’s language preference at risk of being leaked out across any task across most LLMs, highlighting the significance of our attack.

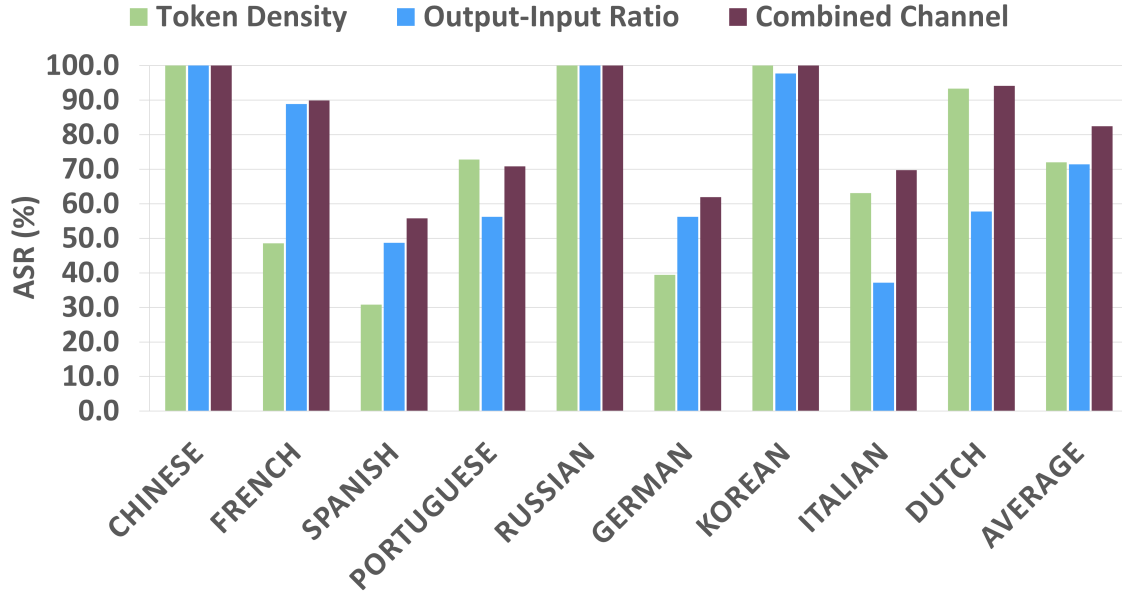


Figure 6.7: Ablation study showing ASR (%) when only output token density or output-input bytes ratio is used as the side channel, compared with attack using combined channel, for Tower model with source language English.

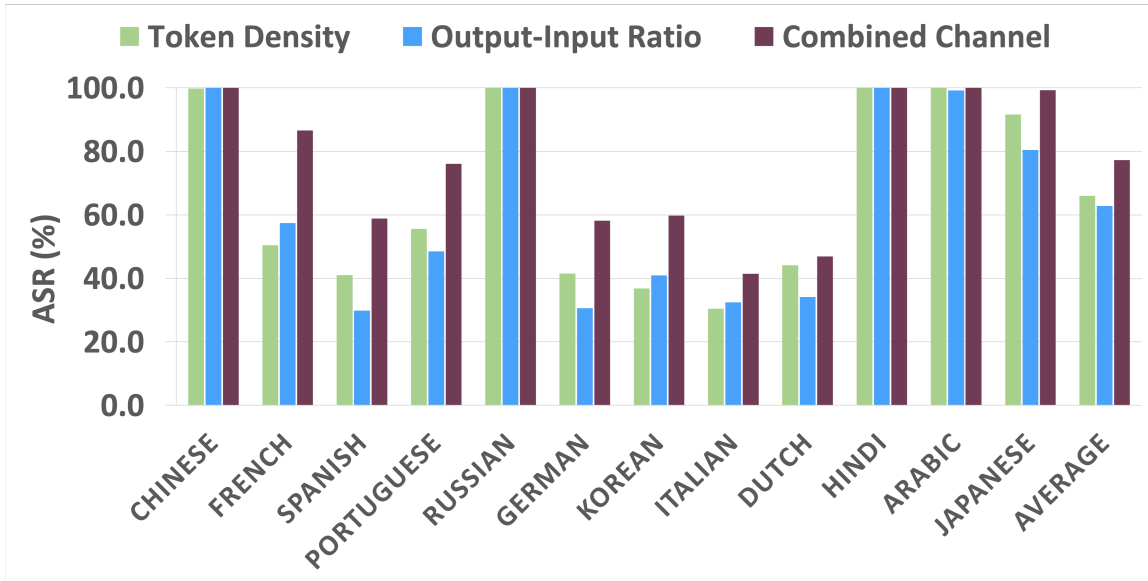


Figure 6.8: Ablation study for ASR (%) using just token density or just output-input length ratio for M2M100 model, versus combined channel (default).

In the GPT-4o setting, the output-input bytes ratio is unaffected by execution time noise and thus plays a more prominent role in attack performance. The output-input bytes ratio alone can achieve an ASR of 73.2%. With the concurrent and median method, the token density channel can reach an ASR of 55.2%. This contributes another 8.4% to reach the final ASR of 81.6%. This highlights the continued relevance of the token density channel in enhancing attack effectiveness even in the presence of excessive execution time noise.

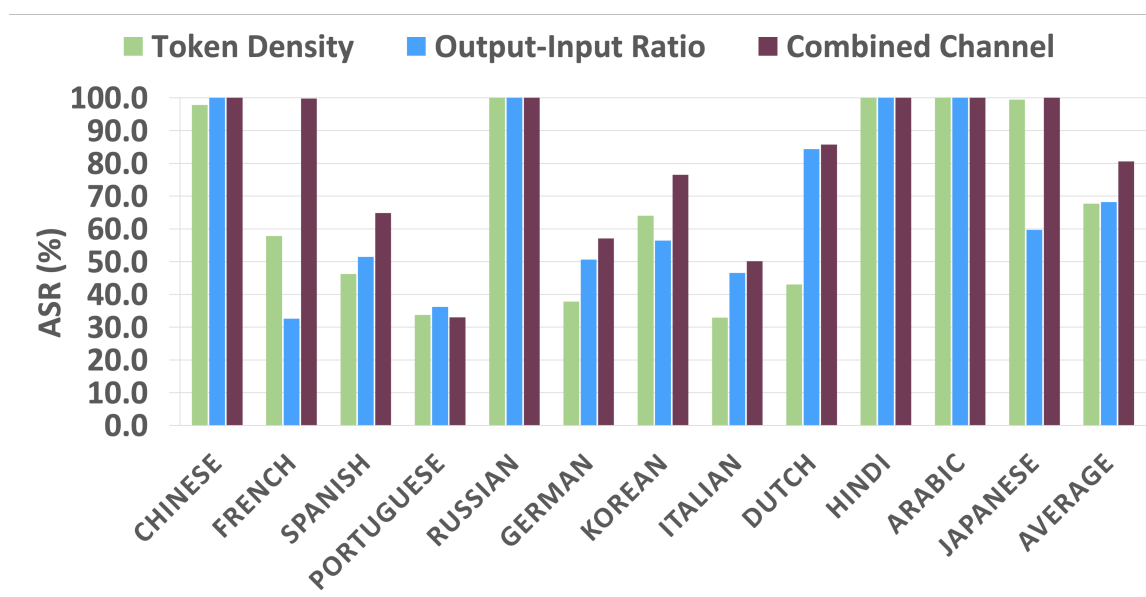


Figure 6.9: Ablation study for ASR (%) with just token density or just output-input length ratio for MBart50, versus combined channel (default).

Chapter 7

Potential Defences

7.1 Prompt-level Defence

For the classification scenario, when providing explanations, one possible solution is to instruct the model to generate a fixed-length output through the prompt. We tested whether this approach could effectively reduce side-channel leakage without altering the demonstration examples by modifying the system prompt from “Provide an explanation” to “Provide a 60-word explanation,” where 60 was selected as the median word count for three-sentence explanations. To evaluate this defence, we pick highly vulnerable scenarios with a high ASR: task227, task391 and task879, using Gemma2-9b-IT and GPT-4o models prompted with augmenting three-shot examples. We assume the attacker has perfect access to the output token count, giving an average ASR with no defence of 79.3% and 88.2%, respectively, for the two models.

Keeping the three-shot examples unchanged, we tested whether modifying the prompt alone could mitigate the side channel. The two models displayed distinct results: the originally more biased GPT-4o experienced a 26% reduction in ASR, dropping to 61.9%, while the Gemma2-9b-IT model maintained a similar level of bias, even with a slight increase in ASR to 80.3%. This experiment demonstrates that the prompt-level defence is only effective for some models, while being completely ineffective for others. A viable option for models that do not respond to this prompt is to use the diminishing few-shot examples to counter the natural bias of the LLM.

7.2 Tokenizer-level Defence

To address the issue of token density differences among languages, it is essential to focus on the root cause, which lies in the tokenization process. Previous efforts to mitigate language bias, such as allocating language capacity based on average log probabilities to optimize downstream performance [54], primarily target performance improvements but do not guarantee consistent token density across languages. To effectively mitigate the timing channel, achieving a more uniform token density is key, thus making the target language anonymous. Despite recent efforts to create more balanced training datasets, intrinsic differences between languages often require varying vocabulary sizes in tokenizers. While changes in the tokenization process may mitigate this side channel, such modifications raise important questions about their potential impact on model performance. We

leave this investigation for future work.

7.3 System-level Defence

In the classification task, explanation lengths vary significantly per prompt, lacking a clear upper bound. Thus, constant-time defence is costly and challenging to get right. With Gemma-9b for 3-shot learning, padding to “maximum” length leads to an average slowdown of 289%, whereas only padding shorter responses to match the “median” length incurs a 40.5% slowdown but still leaks information for longer explanations.

For machine translation, there is no straightforward way to alter the output token count without changing the tokenization methods. However, modifying the tokenization method for existing models typically requires retraining from scratch, which is often impractical. Therefore, one strategy is to pad the output token count and the output plaintext length to match the longest expected output length. With these two metrics obscured, the output-input ratio and output token density will remain at a relatively constant value, effectively mitigating the side channel proposed in Section 6.

For the translation tasks, when delaying the faster responses, the highest latency penalty is observed in Tower, where the language with the highest average token count is Korean, which requires approximately twice as many tokens as other languages due to its unchanged tokenization vocabulary inherited from Llama-2. The average performance cost by padding to match the lowest token density across languages is 93%. In contrast, models like MBart50 and M2M100 exhibit closer average token counts across languages, resulting in an average performance penalty of around 13%.

To achieve a similar level of output byte length, Tower would need to align with the plaintext length of Russian, necessitating an average of 67% padding. MBart50 and M2M100 require an average of 108% padding due to their additional support for Hindi, which has the longest plaintext length.

Chapter 8

Discussion

Impact of dynamic few-shot examples: In the few-shot classification scenario, we use statically chosen examples that do not change based on the user’s input. However, there is a growing trend to use Retrieval Augmented Generation (RAG) to improve model performance by dynamically selecting in-context examples. Under this scenario, we anticipate that if the underlying dataset is biased toward a particular class concerning explanation length, it will produce behaviour similar to that of the biased class prompt. Conversely, an unbiased dataset should perform similarly to the unbiased class prompt. An interesting scenario may arise when the dataset contains outliers that have explanations that are exceedingly long or short. The presence of such outliers may lead to yet another side channel. We leave the exploration of this phenomenon as future work.

Impact of latency and throughput optimizations: For our evaluation, we assume a batch size of 1 to optimize latency and replicate the scenario of a low server load. In real-world systems, applications often aim to achieve low latency and high throughput. One notable batching technique is introduced in Orca [81], which has been further refined into continuous batching that operates on an iteration level. Despite potential noise, we expect the observation channel to remain strong, as responses are still returned at an iteration level that maintains a linear correlation with the output token count. Other optimization techniques, such as speculative decoding, may disrupt the linear relationship between the two variables. However, a rough linear relationship is likely to persist. A promising direction would be to explore how latency optimization techniques might impact this observation channel.

Other vulnerable applications: Beyond the two scenarios evaluated in this work, additional applications may also be susceptible to output token count side-channel leakage. For example, inspired by the setting proposed by Akinsanya et al. [60], where input complexity correlates with specific attributes in adaptive neural networks, reasoning-oriented LLMs may exhibit output lengths that reflect the complexity of reasoning steps, potentially revealing sensitive input attributes such as class labels. The key criterion for identifying output token count side-channel vulnerabilities is whether changes to a specific attribute result in a measurable shift in the distribution of output token counts. Therefore, for LLMs designed for specialized tasks, developers should evaluate whether sensitive attributes can change the output token counts, as this has a risk of information leakage.

Chapter 9

Conclusion

This paper presents the first timing side-channel attack exploiting autoregressive decoding in modern LLMs to leak sensitive user attributes. Our experiments demonstrate a strong correlation between execution time, token count, and sensitive information in specialized LLM tasks. By analyzing LLM timings, attackers can infer the number of generated tokens, thereby uncovering target languages and classification outcomes in applications like machine translation and binary classification. Notably, we demonstrate that our end-to-end network-based timing attacks remain effective even on proprietary models running on remote servers. Finally, we also propose mitigations at various stages, including tokenization, prompt design, and system architecture. With this work, we highlight the need for developers to address variations in token output as a critical vector for information leakage in LLM applications.

Appendix A

Appendix

A.1 Classification Tasks from Natural Instruction Dataset

Table A.1: Classification Tasks from Natural Instructions Dataset Used in Sections 6

Task #	Task Definition	Sample User Input	Sample Output
task145	We would like you to classify each of the following sets of argument pairs (discussing Death Penalty) into either SIMILAR or NOT SIMILAR. A pair of arguments is considered SIMILAR if the arguments are about the same FACET (making the same argument), and is considered NOT SIMILAR if they do not have the same FACET. A FACET is a low-level issue that often reoccurs in many arguments in support of the author's stance or in attacking the other author's position.	Sent1: I am "for" capital punishment... but if a criminal shows a desire for death over a life sentence, I prefer to be sadistic. Sent2: I would prefer to live in a society where no death penalty was handed out cause there was no crime.	Not Similar
task146	We would like you to classify each of the following sets of argument pairs (discussing Gun Control) into either SIMILAR or NOT SIMILAR. A pair of arguments is considered SIMILAR if the arguments are about the same FACET (making the same argument), and is considered NOT SIMILAR if they do not have the same FACET. A FACET is a low-level issue that often reoccurs in many arguments in support of the author's stance or in attacking the other author's position.	Sent1: So now you're saying that just because somebody owns a gun, they're somehow more likely to start killing people than somebody who doesn't own a gun? Sent2: Sure you say guns don't kill people but people kill people, but the gun is only a tool or a means to that end.	Not similar
task147	We would like you to classify each of the following sets of argument pairs (discussing Gay Marriage) into either SIMILAR or NOT SIMILAR. A pair of arguments is considered SIMILAR if the arguments are about the same FACET (making the same argument), and is considered NOT SIMILAR if they do not have the same FACET. A FACET is a low-level issue that often reoccurs in many arguments in support of the author's stance or in attacking the other author's position.	Sent1: I also think that most Americans are truly uninformed when it comes to the issue of same sex marriage. Sent2: I think the real issue is not "should gays be allowed to marry."	Not similar
task227	In this task, you are given an ambiguous question/query (which can be answered in more than one way) and a clarification statement to understand the query more precisely. Your task is to classify whether the given clarification accurately clarifies the given query or not and based on that provide 'Yes' or 'No'.	Query: Tell me about Barbados. Clarification: do you want to file your income tax online	No
task391	In this task, you will be given two sentences separated by ", so". You should decide whether the first sentence can be the cause of the second sentence. If you can see a possible causation, answer with "plausible", otherwise answer with "not plausible".	The terrorist set off the bomb, so they were able to kill him	plausible
task392	In this task, you will be given two sentences separated by "because". You should decide whether the first sentence can be the result of the second sentence. If you can see a possible cause and effect relationship, answer with "plausible", otherwise answer with "not plausible".	I learned how to play the board game because I love to play it	plausible
task577	In this task, you are given a dialogue between a user and an assistant, where users and assistants converse about geographic topics like geopolitical entities and locations. The task here is to find if the dialogue is by the user or assistant. Classify your answers into user and assistant.	Hi. Could you tell me about the culture and society of Pakistan?	user
task590	In this task, you are given an Amazon food product review and its summary. Your task is to generate "True" if the given review and its summary match, otherwise generate "False".	This is great dog food, my dog has severe allergies and this brand is the only one that we can feed. Summary: Great dog food	True
task607	In this task, you are given a text from a social media post. Your task is to classify the given post into two categories: 1) yes if the given post is intentionally offensive, 2) no, otherwise. Also, generate 'no' for offensive statements that appear to be unintentional or non-offensive statements that are underhandedly offensive. Warning: the examples and instances may contain offensive language.	RT @rihanna: I ain't got no type...bad bitches is the only thing that I like	No
task673	Read the given query and classify it as 'Good' or 'Bad' depending on how well the query is formed, 'Bad' being the expected output for a not well-formed query and 'Good' being the expected output for a well-formed query. A query may be wrong based on common sense or general facts, but if it is well-formed, you should answer Good.	What is the torqur for the intake manifold for a 1999 Pontiac Grand Am 3.4 l ?	Good
task679	Classify the given comment into 'Hope Speech' or 'Not Hope Speech'. A hope speech is defined as content that is encouraging, positive or supportive content and talks about equality, diversity or inclusion	Wow this was 4 years ago and nothing changed.	Not Hope Speech
task879	You are given a sentence from a conversation between a human and a virtual assistant. Your task is to identify if the sentence is a question. Answer with Yes or No.	Which city should I search?	Yes

Bibliography

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardaş, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, “Llama 2: Open foundation and fine-tuned chat models,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [3] OpenAI, “Introducing ChatGPT,” 2022. [Online]. Available: <https://openai.com/index/chatgpt/>
- [4] J. Spataro, “Introducing Microsoft 365 Copilot – your copilot for work,” Mar. 2023. [Online]. Available: <https://blogs.microsoft.com/blog/2023/03/16/introducing-microsoft-365-copilot-your-copilot-for-work/>
- [5] Anthropic, “Meet Claude,” 2025. [Online]. Available: <https://www.anthropic.com/claude>
- [6] W. Shi, A. Ajith, M. Xia, Y. Huang, D. Liu, T. Blevins, D. Chen, and L. Zettlemoyer, “Detecting pretraining data from large language models,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [7] M. Meeus, S. Jain, M. Rei, and Y.-A. de Montjoye, “Did the neurons read your book? document-level membership inference for large language models,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 2369–2385.
- [8] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson *et al.*, “Extracting training data from large language models,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2633–2650.

- [9] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847.
- [10] C. Crowell, “20 million predatory loans drained over \$2.4 billion from consumers,” Feb. 2025. [Online]. Available: <https://chicagocrusader.com/20-million-predatory-loans-drained-over-2-4-billion-from-consumers/>
- [11] D. Cameron, “Google Ad-Tech Users Can Target National Security ‘Decision Makers’ and People With Chronic Diseases,” *Wired*, 2025, section: tags. [Online]. Available: <https://www.wired.com/story/google-dv360-banned-audience-segments-national-security/>
- [12] Federal Trade Commision Consumer Advice, “Common Health Scams,” Jan. 2022. [Online]. Available: <https://consumer.ftc.gov/articles/common-health-scams>
- [13] Google Cloud, “Translate text by using adaptive translation.” [Online]. Available: <https://cloud.google.com/translate/docs/advanced/adaptive-translation>
- [14] DeepL Team, “DeepL’s next-gen LLM outperforms ChatGPT-4, Google, and Microsoft for translation quality.” [Online]. Available: <https://www.deepl.com/en/blog/next-gen-language-model>
- [15] European Union, “General Data Protection Regulation - Article 4.” [Online]. Available: <https://gdprinfo.eu/en-article-4>
- [16] M. Braga, “U.S. internet service providers get green light to sell user data — but what about Canada?” *CBC News*, Mar. 2017. [Online]. Available: <https://www.cbc.ca/news/science/us-fcc-internet-privacy-legislation-marketing-ads-canada-1.4046512>
- [17] The Friends Committee on National Legislation, “Impact of Government Surveillance on Muslim Americans and Communities of Color | Friends Committee On National Legislation,” 2017. [Online]. Available: <https://www.fcnl.org/updates/2016-10/impact-government-surveillance-muslim-americans-and-communities-color>
- [18] W. Christou, “Control, censorship and ‘penalties’: inside the Assad regime’s propaganda arm,” *The Guardian*, Dec. 2024. [Online]. Available: <https://www.theguardian.com/world/2024/dec/30/assad-regime-propaganda-arm-syria-control-censorship-penalties>
- [19] R. Weiss, D. Ayzenshteyn, and Y. Mirsky, “What was your prompt? a remote keylogging attack on AI assistants,” in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3367–3384. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/weiss>
- [20] J. Wei, A. Abdulrazzag, T. Zhang, A. Muursepp, and G. Saileshwar, “Privacy risks of speculative decoding in large language models,” *arXiv preprint arXiv:2411.01076*, 2024.
- [21] L. Song, Z. Pang, W. Wang, Z. Wang, X. Wang, H. Chen, W. Song, Y. Jin, D. Meng, and R. Hou, “The early bird catches the leak: Unveiling timing side channels in llm serving systems,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.20002>

- [22] A. Petrov, E. La Malfa, P. Torr, and A. Bibi, “Language Model Tokenizers Introduce Unfairness Between Languages,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Nau-
mann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 36 963–36 990.
- [23] O. Ahia, S. Kumar, H. Gonen, J. Kasai, D. R. Mortensen, N. A. Smith, and Y. Tsvetkov, “Do
all languages cost the same? tokenization in the era of commercial language models,” 2023.
- [24] P. C. Kocher, “Timing attacks on implementations of diffie-hellman, rsa, dss, and other sys-
tems,” in *Advances in Cryptology — CRYPTO ’96*, N. Koblitz, Ed. Berlin, Heidelberg:
Springer Berlin Heidelberg, 1996, pp. 104–113.
- [25] B. B. Brumley and N. Tuveri, “Remote timing attacks are still practical,” in *Computer Security
– ESORICS 2011*, V. Atluri and C. Diaz, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg,
2011, pp. 355–371.
- [26] P. Kocher, J. Jaffe, and B. Jun, “Differential power analysis,” in *Advances in Cryptology —
CRYPTO’ 99*, M. Wiener, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp.
388–397.
- [27] J.-J. Quisquater and D. Samyde, “Electromagnetic analysis (ema): Measures and counter-
measures for smart cards,” in *Smart Card Programming and Security*, I. Attali and T. Jensen,
Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 200–210.
- [28] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, “Last-level cache side-channel attacks are
practical,” in *2015 IEEE Symposium on Security and Privacy*, 2015, pp. 605–622.
- [29] Y. Yarom and K. Falkner, “Flush+reload: a high resolution, low noise, l3 cache side-channel
attack,” in *Proceedings of the 23rd USENIX Conference on Security Symposium*, ser. SEC’14.
USA: USENIX Association, 2014, p. 719–732.
- [30] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard,
P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, “Meltdown: Reading kernel
memory from user space,” in *27th USENIX Security Symposium (USENIX Security
18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 973–990. [Online]. Available:
<https://www.usenix.org/conference/usenixsecurity18/presentation/lipp>
- [31] J. V. Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein,
T. F. Wenisch, Y. Yarom, and R. Strackx, “Foreshadow: Extracting the keys to the intel
SGX kingdom with transient Out-of-Order execution,” in *27th USENIX Security Symposium
(USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, p. 991–1008.
[Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/bulck>
- [32] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard,
T. Prescher, M. Schwarz, and Y. Yarom, “Spectre attacks: Exploiting speculative execution,”
in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 1–19.
- [33] H. Naghibijouybari, A. Neupane, Z. Qian, and N. Abu-Ghazaleh, “Rendered insecure:
Gpu side channel attacks are practical,” in *Proceedings of the 2018 ACM SIGSAC*

- Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 2139–2153. [Online]. Available: <https://doi.org/10.1145/3243734.3243831>
- [34] J. Wei, Y. Zhang, Z. Zhou, Z. Li, and M. A. Al Faruque, “Leaky dnn: Stealing deep-learning model secret with gpu context-switching side-channel,” in *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2020, pp. 125–137.
- [35] M. Yan, C. W. Fletcher, and J. Torrellas, “Cache telepathy: Leveraging shared resource attacks to learn DNN architectures,” in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 2003–2020. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/yan>
- [36] X. Hu, L. Liang, S. Li, L. Deng, P. Zuo, Y. Ji, X. Xie, Y. Ding, C. Liu, T. Sherwood, and Y. Xie, “Deepsniffer: A dnn model extraction framework based on learning architectural hints,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 385–399. [Online]. Available: <https://doi.org/10.1145/3373376.3378460>
- [37] E. Debenedetti, G. Severi, N. Carlini, C. A. Choquette-Choo, M. Jagielski, M. Nasr, E. Wallace, and F. Tramèr, “Privacy side channels in machine learning systems,” in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 6861–6848. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/debenedetti>
- [38] OpenAI, “Hello GPT-4o,” 2024. [Online]. Available: <https://openai.com/index/hello-gpt-4o/>
- [39] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186.
- [40] Q. Chen, Z. Zhuo, and W. Wang, “Bert for joint intent classification and slot filling,” *arXiv preprint arXiv:1902.10909*, 2019.
- [41] M. Grootendorst, “Bertopic: Neural topic modeling with a class-based tf-idf procedure,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.05794>
- [42] J. Howard and S. Ruder, “Universal language model fine-tuning for text classification,” in *ACL 2018-56th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference (Long Papers)*, vol. 1. Association for Computational Linguistics, 2018, pp. 328–339.
- [43] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and

- D. Amodei, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (Neurips)*, 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [44] T. Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, “Calibrate before use: Improving few-shot performance of language models,” 2021. [Online]. Available: <https://arxiv.org/abs/2102.09690>
- [45] X. Ye and G. Durrett, “The unreliability of explanations in few-shot prompting for textual reasoning,” 2022. [Online]. Available: <https://arxiv.org/abs/2205.03401>
- [46] F. Nie, M. Chen, Z. Zhang, and X. Cheng, “Improving few-shot performance of language models via nearest neighbor calibration,” 2022. [Online]. Available: <https://arxiv.org/abs/2212.02216>
- [47] H. Ma, C. Zhang, Y. Bian, L. Liu, Z. Zhang, P. Zhao, S. Zhang, H. Fu, Q. Hu, and B. Wu, “Fairness-guided few-shot prompting for large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.13217>
- [48] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2016, pp. 1715–1725.
- [49] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, J. Klingner, A. Shah, M. Johnson, X. Liu, Lukasz Kaiser, S. Gouws, Y. Kato, T. Kudo, H. Kazawa, K. Stevens, G. Kurian, N. Patil, W. Wang, C. Young, J. Smith, J. Riesa, A. Rudnick, O. Vinyals, G. Corrado, M. Hughes, and J. Dean, “Google’s neural machine translation system: Bridging the gap between human and machine translation,” 2016. [Online]. Available: <https://arxiv.org/abs/1609.08144>
- [50] T. Kudo and J. Richardson, “Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2018, pp. 66–71.
- [51] O. Goldman, A. Caciularu, M. Eyal, K. Cao, I. Szpektor, and R. Tsarfaty, “Unpacking tokenization: Evaluating text compression and its correlation with model performance,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.06265>
- [52] A. Fan, S. Bhosale, H. Schwenk, Z. Ma, A. El-Kishky, S. Goyal, M. Baines, O. Celebi, G. Wenzek, V. Chaudhary, N. Goyal, T. Birch, V. Liptchinsky, S. Edunov, E. Grave, M. Auli, and A. Joulin, “Beyond english-centric multilingual machine translation,” 2020. [Online]. Available: <https://arxiv.org/abs/2010.11125>
- [53] Big Science Workshop, “Bloom: A 176b-parameter open-access multilingual language model,” 2023. [Online]. Available: <https://arxiv.org/abs/2211.05100>
- [54] B. Zheng, L. Dong, S. Huang, S. Singhal, W. Che, T. Liu, X. Song, and F. Wei, “Allocating large vocabulary capacity for cross-lingual language model pre-training,” 2021. [Online]. Available: <https://arxiv.org/abs/2109.07306>

- [55] H. Lian, Y. Xiong, J. Niu, S. Mo, Z. Su, Z. Lin, P. Liu, H. Chen, and G. Ding, “Scaffold-bpe: Enhancing byte pair encoding with simple and effective scaffold token removal,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.17808>
- [56] D. Liang, H. Gonen, Y. Mao, R. Hou, N. Goyal, M. Ghazvininejad, L. Zettlemoyer, and M. Khabsa, “Xlm-v: Overcoming the vocabulary bottleneck in multilingual masked language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2301.10472>
- [57] V. Duddu, D. Samanta, D. V. Rao, and V. E. Balas, “Stealing neural networks via timing side channels,” 2019. [Online]. Available: <https://arxiv.org/abs/1812.11720>
- [58] R. S. Ali, B. Z. H. Zhao, H. J. Asghar, T. Nguyen, I. D. Wood, and D. Kaafar, “Unintended memorization and timing attacks in named entity recognition models,” 2022. [Online]. Available: <https://arxiv.org/abs/2211.02245>
- [59] S. Shukla, M. Alam, S. Bhattacharya, D. Mukhopadhyay, and P. Mitra, “On the evaluation of user privacy in deep neural networks using timing side channel,” 2024. [Online]. Available: <https://arxiv.org/abs/2208.01113>
- [60] A. Akinsanya and T. Brennan, “Timing channels in adaptive neural networks,” in *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society, 2024. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/timing-channels-in-adaptive-neural-networks/>
- [61] M. Soleimani, G. Jia, I. Gim, S. seob Lee, and A. Khandelwal, “Wiretapping LLMs: Network side-channel attacks on interactive LLM services,” Cryptology ePrint Archive, Paper 2025/167, 2025. [Online]. Available: <https://eprint.iacr.org/2025/167>
- [62] X. Zheng, H. Han, S. Shi, Q. Fang, Z. Du, X. Hu, and Q. Guo, “Inputsntatch: Stealing input in llm services via timing side-channel attacks,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.18191>
- [63] N. Carlini and M. Nasr, “Remote timing attacks on efficient language model inference,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.17175>
- [64] Kasisto, “Kasisto Launches KAI-GPT, the First Banking Industry-Specific Large Language Model,” 2023. [Online]. Available: <https://www.businesswire.com/news/home/20230531005140/en/Kasisto-Launches-KAI-GPT-the-First-Banking-Industry-Specific-Large-Language-Model>
- [65] Google Research, “Med-PaLM: A Medical Large Language Model - Google Research.” [Online]. Available: <https://sites.research.google/med-palm/>
- [66] G. Fahner, “FICO Score Research: Explainable AI for Credit Scoring,” Jan. 2019. [Online]. Available: <https://www.fico.com/blogs/fico-score-research-explainable-ai-credit-scoring>
- [67] Equifax, “Explainability and artificial intelligence,” 2024. [Online]. Available: <https://www.equifax.com/newsroom/all-news/-/story/explainability-and-artificial-intelligence>

- [68] C. Ikezuruora, “Demystifying the Right to Explanation Under GDPR: A Comprehensive Guide,” Dec. 2023. [Online]. Available: <https://www.privacyend.com/right-to-explanation-under-gdpr/>
- [69] Y. Wang, S. Mishra, P. Alipoormolabashi, Y. Kordi, A. Mirzaei, A. Arunkumar, A. Ashok, A. S. Dhanasekaran, A. Naik, D. Stap *et al.*, “Super-naturalinstructions:generalization via declarative instructions on 1600+ tasks,” in *EMNLP*, 2022.
- [70] OpenAI, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [71] Gemma Team, “Gemma 2: Improving open language models at a practical size,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.00118>
- [72] AI@Meta, “Llama 3 model card,” 2024. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [73] T.-w. Lo, “Loan Approval Classification Dataset,” 2024. [Online]. Available: <https://www.kaggle.com/datasets/taweilo/loan-approval-classification-data>
- [74] Abod0x, “SymptomsXDiagnosis (SXD),” 2024. [Online]. Available: <https://www.kaggle.com/datasets/abod0x/symptomsxdiagnose-sxd>
- [75] A. P. Dempster, N. M. Laird, and D. B. Rubin, “Maximum likelihood from incomplete data via the em algorithm,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 39, no. 1, pp. 1–22, 1977. [Online]. Available: <https://rss.onlinelibrary.wiley.com/doi/abs/10.1111/j.2517-6161.1977.tb01600.x>
- [76] A. Bhattacharyya, “On a measure of divergence between two multinomial populations,” *Sankhyā: The Indian Journal of Statistics (1933-1960)*, vol. 7, no. 4, pp. 401–406, 1946. [Online]. Available: <http://www.jstor.org/stable/25047882>
- [77] Y. Tang, C. Tran, X. Li, P.-J. Chen, N. Goyal, V. Chaudhary, J. Gu, and A. Fan, “Multilingual translation with extensible multilingual pretraining and finetuning,” 2020. [Online]. Available: <https://arxiv.org/abs/2008.00401>
- [78] D. M. Alves, J. Pombal, N. M. Guerreiro, P. H. Martins, J. Alves, A. Farajian, B. Peters, R. Rei, P. Fernandes, S. Agrawal, P. Colombo, J. G. C. de Souza, and A. F. T. Martins, “Tower: An open multilingual large language model for translation-related tasks,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.17733>
- [79] NLLB Team, “No language left behind: Scaling human-centered machine translation,” 2022. [Online]. Available: <https://arxiv.org/abs/2207.04672>
- [80] P. Koehn, “Europarl: A parallel corpus for statistical machine translation,” in *Proceedings of Machine Translation Summit X: Papers*, Phuket, Thailand, Sep. 13-15 2005, pp. 79–86. [Online]. Available: <https://aclanthology.org/2005.mtsummit-papers.11>
- [81] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/yu>

ProQuest Number: 32236290

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by
ProQuest LLC a part of Clarivate (2025).
Copyright of the Dissertation is held by the Author unless otherwise noted.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

This work may be used in accordance with the terms of the Creative Commons license
or other rights statement, as indicated in the copyright statement or in the metadata
associated with this work. Unless otherwise specified in the copyright statement
or the metadata, all rights are reserved by the copyright holder.

ProQuest LLC
789 East Eisenhower Parkway
Ann Arbor, MI 48108 USA